

Machine Learning

Lecture 10: Dimensionality Reduction & Matrix Factorization

Prof. Dr. Stefan Leutenegger

Smart Robotics Lab
Technical University of Munich

Winter term 2024/2025

Chapter: Dimensionality Reduction & Matrix Factorization

1. **Introduction**
2. Principal Component Analysis (PCA)
3. Singular Value Decomposition (SVD)
4. Matrix Factorization
5. Neighbor Graph Methods
6. Autoencoders (Nonlinear Dimensionality Reduction)

Introduction: Unsupervised Learning

- Supervised learning aims to map inputs to targets with $\mathbf{y} = f(\mathbf{x})$, or in a probabilistic framework to model $p(\mathbf{y}|\mathbf{x})$
- Unsupervised learning can be seen as modeling $p(\mathbf{x})$
- We are trying to find the (hidden/latent) structure in the data
 - e.g find a latent distribution $p(\mathbf{z})$ and a generative transformation $p(\mathbf{x}|\mathbf{z})$ to obtain $p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z}$
 - Latent $\mathbf{z}$ usually unknown and has to be estimated
- Examples:
 - Clustering: The cluster label is the latent state
 - Anomaly detection: treat instances with low $p(\mathbf{x})$ as anomalies

Introduction: Unsupervised Learning

- Unsupervised learning can be viewed as compression
 - Compress a data point to a single label corresponding to its cluster
 - Compress a data point from a higher to a lower dim. latent space
- Unsupervised learning can be used:
 - As standalone method (e.g. to understand your data, visualization)
 - As pre-processing step (e.g. use cluster label as feature for subsequent classification task; obtain small number of relevant features)
 - To leverage large amounts of unlabeled data for pretraining
- This lecture: Dimensionality Reduction (& Matrix Factorization)

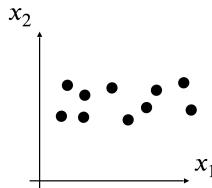
Dimensionality Reduction: Motivation

- Often data has very many features, i.e. high-dimensional data
 - Similarity search/computation is expensive because of high complexity of distance functions
 - Highly correlated dimensions could cause trouble for some algorithms
 - Curse of dimensionality: exponential amounts of data needed to characterize the density as the dim. goes up
 - It is hard to visualize high-dimensional data
- Often it lies on a low-dim. manifold, embedded in a high-dim. space
 - Goal: Try to reduce the dim. while avoiding information loss
 - Benefits:
 - Computational or memory savings
 - Uncover the intrinsic dimensionality of the data
 - (more benefits later...)

Feature (Sub-)Selection

Choose "good" dimensions using a-priori knowledge or appropriate heuristics

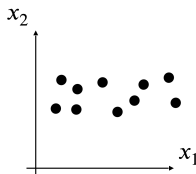
- e.g. remove **low-variance** dimensions
- Depending on the application only a few dimensions might be of interest
 - Example: shoe size interesting for shoe purchases, not so for car purchases
- Advantage:
 - No need for an intensive preprocessing or training phase to determine relevant dimensions
- Disadvantage:
 - Expert knowledge required; misjudgment possible
 - Univariate feature selection ignores correlations



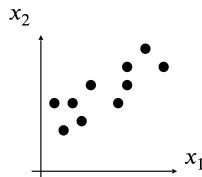
Beyond Feature (Sub-)Selection

Can we do

- better?
- automatic?

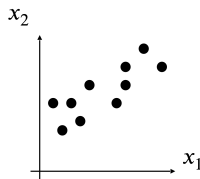


VS.

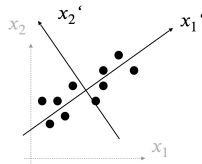


Obviously: Simply discarding whole features not a good idea

- Features are often correlated



*transformation
of coordinate
axes*

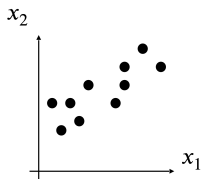


*The x_2' dimension
is barely necessary
to describe the
data in this new
coordinate system
→ can be ignored*

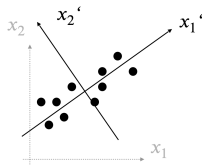
Dim. Reduction via Linear Transformations

- Use linear transformations to represent data in a different coordinate system
 - **Change of basis** (orthogonal basis transformations) and **potentially discarding dimensions**
- Technical:
 - Use orthonormal transformation matrix $F \in \mathbb{R}^{d \times k}$
 - $(x')^T = x^T \cdot F$ is the transformation of (column) vector x into the new coordinate system defined by F
 - $X' = X \cdot F$ is the matrix containing all the transformed points x'_i

$$\begin{matrix} \boxed{N \times d} \\ \cdot \end{matrix} \begin{matrix} \boxed{d \times k} \\ \cdot \end{matrix} = \begin{matrix} \boxed{N \times k} \end{matrix}$$



*(first center to mean, then)
transformation
with F*



Discussion: Linear Transformations

- Feature selection is a linear transformation
 - What is matrix F ?
- Let $\bar{x}$ be the mean vector (here: row vector) in the original data space, the mean vector in the transformed space is given by $\bar{x}' = \bar{x} \cdot F$
 - Proof?
- Let Σ_X be the covariance matrix in the original data space, the covariance matrix in the transformed space is then $\Sigma_{X'} = F^T \cdot \Sigma_X \cdot F$
 - Proof?

Chapter: Dimensionality Reduction & Matrix Factorization

1. Introduction
2. **Principal Component Analysis (PCA)**
3. Singular Value Decomposition (SVD)
4. Matrix Factorization
5. Neighbor Graph Methods
6. Autoencoders (Nonlinear Dimensionality Reduction)

Principal Component Analysis: Motivation

- Question: Which transformation matrix F to use?
 - Is there an **optimal orthogonal transformation** (depending on the data)?
 - Optimality: Approximate the data with few coefficients as well as possible



- Approach: Principal Component Analysis (PCA)
 - Find a coordinate system in which the (possibly originally correlated) points are **linearly uncorrelated**
 - The dimensions with no or low variance can then be ignored

Determine the Principal Components

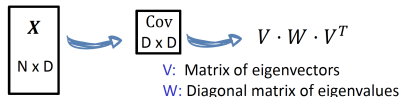
- Goal:
 - Transform the data, such that the **covariance between the new dimensions is 0**
 - The transformed data points are not linearly correlated anymore
- Given: N d -dimensional data points:
 $\{\mathbf{x}_i\}_{i=1}^N, \mathbf{x}_i \in \mathbb{R}^d \quad \forall i \in \{1, \dots, N\}$
- We represent this set of points by a matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$:

$$\mathbf{X} = \begin{bmatrix} x_{11} & \dots & x_{1d} \\ \vdots & \ddots & \vdots \\ x_{N1} & \dots & x_{Nd} \end{bmatrix}$$

- The row $\mathbf{x}_i = \{x_{i1}, \dots, x_{id}\} \in \mathbb{R}^d$ denotes the i -th point and the column $\mathbf{X}_{:,j}$ denotes the vector containing all values from the j -th dimension

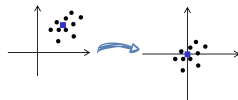
Determine the Principal Components

- Goal:
 - Transform the data, such that the **covariance between the new dimensions is 0**
 - The transformed data points are not linearly correlated anymore
- General approach
 1. Center the data
 2. Compute the covariance matrix
 3. Use the Eigenvector Decomposition to transform the coordinate system



Determine the Principal Components

- Given: $\mathbf{X} \in \mathbb{R}^{N \times d}$: $\mathbf{X} = \begin{bmatrix} x_{11} & \dots & x_{1d} \\ \vdots & \ddots & \vdots \\ x_{N1} & \dots & x_{Nd} \end{bmatrix}$
- Shift the points by their mean $\bar{\mathbf{x}} \in \mathbb{R}^d$ (centralized data): $\tilde{\mathbf{x}}_i = \mathbf{x}_i - \bar{\mathbf{x}}$



Statistics:

Zero-order statistic: number of points N

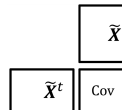
First-order statistic: mean of the N points, the vector $\bar{\mathbf{x}} \in \mathbb{R}^d$:

$$\bar{\mathbf{x}} = \begin{bmatrix} \bar{x}_1 \\ \vdots \\ \bar{x}_d \end{bmatrix} = \frac{1}{N} \cdot \mathbf{X}^T \cdot \mathbf{1}_N \quad (1)$$

where $\mathbf{1}_N$ is an N -dimensional vector of ones

Determine the Principal Components

- Determine the variances $Var(\tilde{\mathbf{X}}_j)$ for each dimension $j \in \{1, \dots, d\}$
- Determine the covariance $Cov(\tilde{\mathbf{X}}_{j_1}, \tilde{\mathbf{X}}_{j_2})$ between dimensions j_1 and j_2 , $\forall j_1 \neq j_2 \in \{1, \dots, d\}$
- Leads to covariance matrix $\Sigma_{\tilde{\mathbf{X}}} \in \mathbb{R}^{d \times d}$



Second-order statistic: variance and covariance

The variance within the j -th dimension in $\mathbf{X}$ is:

$$Var(\mathbf{X}_j) = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_{ij} - \bar{\mathbf{x}}_j)^2 = \frac{1}{N} \mathbf{X}_j^T \mathbf{X}_j - \bar{\mathbf{x}}_j^2 \quad (2)$$

The covariance between dimensions j_1 and j_2 is:

$$Cov(\mathbf{X}_{j_1}, \mathbf{X}_{j_2}) = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_{ij_1} - \bar{\mathbf{x}}_{j_1})(\mathbf{x}_{ij_2} - \bar{\mathbf{x}}_{j_2}) = \frac{1}{N} \mathbf{X}_{j_1}^T \mathbf{X}_{j_2} - \bar{\mathbf{x}}_{j_1} \bar{\mathbf{x}}_{j_2} \quad (3)$$

Determine the Principal Components

Statistics:

For the set of points contained in $\mathbf{X}$ the corresponding covariance matrix is defined as:

$$\Sigma_{\mathbf{X}} = \begin{bmatrix} \text{Var}(\mathbf{X}_1) & \text{Cov}(\mathbf{X}_1, \mathbf{X}_2) & \dots & \text{Cov}(\mathbf{X}_1, \mathbf{X}_d) \\ \text{Cov}(\mathbf{X}_2, \mathbf{X}_1) & \text{Var}(\mathbf{X}_2) & & \\ \vdots & & \ddots & \vdots \\ \text{Cov}(\mathbf{X}_d, \mathbf{X}_1) & \dots & & \text{Var}(\mathbf{X}_d) \end{bmatrix} = \frac{1}{N} \mathbf{X}^T \mathbf{X} - \hat{\mathbf{X}} \hat{\mathbf{X}}^T \quad (4)$$

Remark: Covariance matrices are symmetric

Determine the Principal Components

Goal of PCA: Transformation of the coordinate system such that the covariances between the new axes are 0

$$\Lambda = \begin{bmatrix} Var(1)' & 0 & \dots & 0 \\ 0 & Var(2)' & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & Var(D)' \end{bmatrix}$$

- Approach:
 - Diagonalization by changing the basis (= adapt coordinate system)
 - According to the spectral theorem, the eigenvectors of a symmetric matrix form an orthogonal basis
- Eigendecomposition of the covariance matrix: $\Sigma_{\tilde{X}} = \Gamma \cdot \Lambda \cdot \Gamma^T$

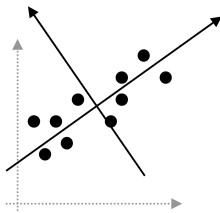
Eigen(-spectral)decomposition is the factorization of $\mathbf{X} \in \mathbb{R}^{d \times d}$:

$$\mathbf{X} = \Gamma \cdot \Lambda \cdot \Gamma^T \quad (5)$$

- Matrices $\Gamma, \Lambda \in \mathbb{R}^{d \times d}$ with columns of Γ being the normalized eigenvectors γ_i
- Γ is an orthonormal matrix: $\Gamma \cdot \Gamma^T = \Gamma^T \cdot \Gamma = \text{Id}(\Gamma^T = \Gamma^{-1})$
- Λ is a diagonal matrix with eigenvalues λ_i as the diagonal elements

Determine the Principal Components

- Eigendecomposition of the covariance matrix: $\Sigma_{\tilde{X}} = \Gamma \cdot \Lambda \cdot \Gamma^T$
- The **new coordinate system** is defined by the eigenvectors γ_i :
 - Transformed data: $Y = \tilde{X} \cdot \Gamma$
 - Λ is the covariance matrix in this new coordinate system
 - New system has variance λ_i in dimension i
 - $\forall i_1 \neq i_2 : Cov(Y_{i_1}, Y_{i_2}) = 0$



Dimensionality Reduction with PCA

- Approach
 - The coordinates with low variance (hence low λ_i) can be ignored
 - W.l.o.g. let us assume $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$
- Truncation of $\mathbf{\Gamma}$
 - **Keep only columns (i.e. eigenvectors) of $\mathbf{\Gamma}$ corresponding to the largest k eigenvalues $\lambda_1, \dots, \lambda_k$**
 - $\mathbf{Y}_{reduced} = \tilde{\mathbf{X}} \cdot \mathbf{\Gamma}_{truncated}$
- How to pick k ?
 - Frequently used: 90% rule; the k variances should explain 90% of the energy
 - $k =$ smallest value ensuring $\sum_{i=1}^k \lambda_i \geq 0.9 \cdot \sum_{i=1}^d \lambda_i$
- The modified points (transformed and truncated) contain most of the information of the original points and are low-dimensional

- Complexity of PCA:

$$\underbrace{O(N \cdot d^2)}_{\text{Computing covariance matrix}} + \underbrace{O(d^3)}_{\text{Eigenvalue decomposition}} + \underbrace{O(N \cdot d \cdot k)}_{\text{Projection onto k-dimensional space}} = O(N \cdot d^2 + d^3)$$

- Remarks on eigenvalue decomposition:
 - Usually we are interested in the reduced data only
 - **Only the k largest eigenvectors required** (i.e. not all of them)
 - Use iterative approaches (next slide) for finding eigenvectors
 - Complexity: $O(k \cdot d^2)$
 - For sparse data even faster: $O(k \cdot \underbrace{\#nz}_{\text{number of nonzero elements}})$

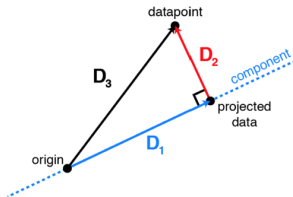
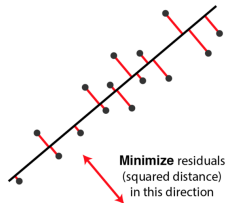
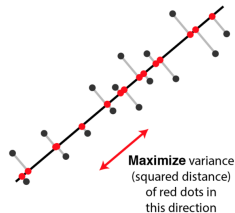
How to Compute Eigenvectors?

- Eigenvalues are important for many machine learning tasks
 - PCA, Ranking of Websites, Community Detection, . . . //see our other lecture!
 - How to compute them efficiently?
- Power iteration (a.k.a. Von Misesiteration)
 - Iterative approach to compute **a single** eigenvector
- Let $\mathbf{A}$ be a matrix and $\mathbf{v}$ be an arbitrary (normalized) vector
 - Iteratively compute $\mathbf{v} \leftarrow \frac{\mathbf{A} \cdot \mathbf{v}}{\|\mathbf{A} \cdot \mathbf{v}\|}$
 - In each step, $\mathbf{v}$ is simply multiplied with $\mathbf{A}$ and normalized
 - $\mathbf{v}$ **converges to the eigenvector of $\mathbf{A}$ with corresponding eigenvalue of largest absolute value**
 - Highly efficient for sparse data

How to Compute Eigenvectors?

- Converges
 - Linear convergence with rate $|\lambda_2/\lambda_1|$
 - Fast convergence if first and second eigenvalue are dissimilar
- How to find **multiple (the k largest) eigenvectors?**
 - Let us focus on symmetric matrices A
 - Eigenvalue decomposition leads to: $A = \Gamma \cdot \Lambda \cdot \Gamma^T = \sum_{i=1}^d \lambda_i \cdot \gamma_i \cdot \gamma_i^T$
 - Define deflated matrix: $\tilde{A} = A - \lambda_1 \cdot \gamma_1 \cdot \gamma_1^T$
 - $\tilde{A}$ has the same eigenvectors as A except the first one has become zero
 - Apply power iteration on $\tilde{A}$ to find the second largest eigenvector of A

Alternative views of PCA



Images adapted from Alex H. Williams

PCA vs. Regression

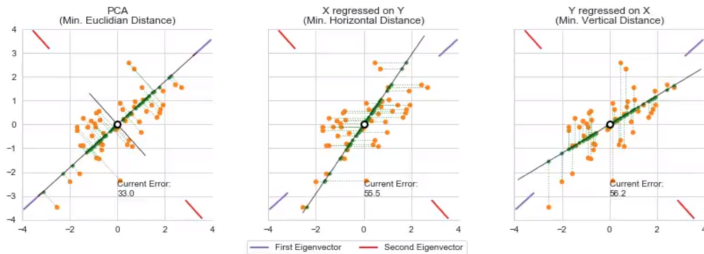


Image adapted from Quentin André

PCA: Summary

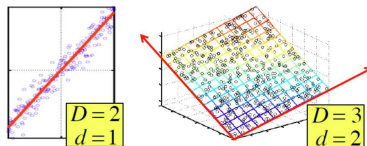
- PCA finds the optimal transformation by deriving uncorrelated dimensions
 - Exploits eigendecomposition
- Dimensionality reduction
 - After transformation simply remove dimensions with lowest variance (or use only the k largest eigenvectors for transformation)
- Limitations
 - Only captures linear relationships (one solution: Kernel PCA)

Chapter: Dimensionality Reduction & Matrix Factorization

1. Introduction
2. Principal Component Analysis (PCA)
3. **Singular Value Decomposition (SVD)**
 - **Idea: Low Rank Approximation**
 - SVD & Latent Factors
 - Dimensionality Reduction
4. Matrix Factorization
5. Neighbor Graph Methods
6. Autoencoders (Nonlinear Dimensionality Reduction)

Low-Dimensional Manifold

- Data often lies on a low-dimensional manifold embedded in higher-dimensional space



- How can we measure the dimensionality of this manifold?
 - Put differently: how to measure the intrinsic dimensionality of the data
- How can we find this manifold?

Rank of a Matrix

- Q: What is the rank of a matrix A ?
- A: Number of linearly independent columns/rows of A
- Example:

- Matrix $A = \begin{bmatrix} 1 & 2 & 1 \\ -2 & -3 & 1 \\ 3 & 5 & 0 \end{bmatrix}$ has a rank $r = 2$

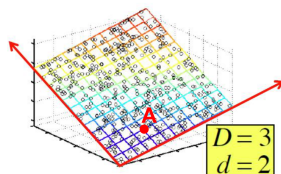
Why? The first two rows are linearly independent, so the rank is at least 2, but all three rows are linearly dependent (the first is equal to the sum of the second and third) so the rank must be less than 3.

- Why do we care about low rank?
 - We can write A as two "basis" vectors: $\begin{bmatrix} 1 & 2 & 1 \end{bmatrix}, \begin{bmatrix} -2 & -3 & -1 \end{bmatrix}$
 - And new coordinates of: $\begin{bmatrix} 1 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & -1 \end{bmatrix}$

Rank is "Dimensionality"

- Cloud of points in 3D space:
 - Think of point positions as a matrix:

1 row per point:
$$\begin{pmatrix} 1 & 2 & 1 \\ -2 & -3 & 1 \\ 3 & 5 & 0 \end{pmatrix} \begin{matrix} A \\ B \\ C \end{matrix}$$



- **We can rewrite coordinates more efficiently!**
 - Old basis vectors: $[1 \ 0 \ 0], [0 \ 1 \ 0], [0 \ 0 \ 1]$
 - **New basis vectors:** $[1 \ 2 \ 1], [-2 \ -3 \ -1]$
 - Then A has new coordinates: $[1 \ 0]$, B: $[0 \ 1]$, C: $[1 \ -1]$
 - **Notice: We reduced the number of coordinates!**

Low Rank Approximation

- Idea: approximate original data $\mathbf{A}$ by a low rank matrix $\mathbf{B}$

$$\mathbf{A} = \begin{bmatrix} 1.01 & 2.05 & 0.9 \\ -2.1 & -3.05 & 1.1 \\ 2.99 & 5.01 & 0.3 \end{bmatrix} \approx \begin{bmatrix} 1 & 2 & 1 \\ -2 & -3 & 1 \\ 3 & 5 & 0 \end{bmatrix} = \mathbf{B}$$

$$\text{rank}(\mathbf{A}) = 3$$

We need 3 coordinates
to describe each point

$$\text{rank}(\mathbf{B}) = 2$$

We need only 2 coordinates
per point

Low Rank Approximation

- Idea: approximate original data $\mathbf{A}$ by a low rank matrix $\mathbf{B}$

$$\mathbf{A} = \begin{bmatrix} 1.01 & 2.05 & 0.9 \\ -2.1 & -3.05 & 1.1 \\ 2.99 & 5.01 & 0.3 \end{bmatrix} \approx \begin{bmatrix} 1 & 2 & 1 \\ -2 & -3 & 1 \\ 3 & 5 & 0 \end{bmatrix} = \mathbf{B}$$

- Important: Even though both $\mathbf{A}$ and $\mathbf{B}$ are $\in \mathbb{R}^{n \times d}$ we need only two coordinates per point to describe $\mathbf{B}$
 - $\text{rank}(\mathbf{A}) = 3$ vs. $\text{rank}(\mathbf{A}) = 2$ (3 vs. 2 coordinates per point)
- Goal: Find the best low-rank approximation**
 - best = minimize the sum of reconstruction error
 - Given matrix $\mathbf{A} \in \mathbb{R}^{n \times d}$, find $\mathbf{B} \in \mathbb{R}^{n \times d}$ with $\text{rank}(\mathbf{B}) = k$ that minimizes

$$\|\mathbf{A} - \mathbf{B}\|_F^2 = \sum_{i=1}^N \sum_{j=1}^D (a_{ij} - b_{ij})^2 \quad (6)$$

Chapter: Dimensionality Reduction & Matrix Factorization

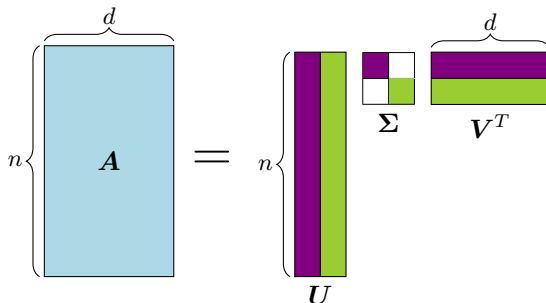
1. Introduction
2. Principal Component Analysis (PCA)
3. **Singular Value Decomposition (SVD)**
 - Idea: Low Rank Approximation
 - **SVD & Latent Factors**
 - Dimensionality Reduction
4. Matrix Factorization
5. Neighbor Graph Methods
6. Autoencoders (Nonlinear Dimensionality Reduction)

Singular Value Decomposition (SVD): Definition

- Each real matrix $A \in \mathbb{R}^{n \times d}$ can be decomposed into $A = U \cdot \Sigma \cdot V^T$ (note: **exact representation**, not approximation), where
 - $U \in \mathbb{R}^{n \times r}$, $\Sigma \in \mathbb{R}^{r \times r}$, $V \in \mathbb{R}^{d \times r}$
 - U, V : column orthonormal
 - i.e. $U^T U = I$; $V^T V = I$ (I identity matrix)
 - U are called the left singular vectors, V the right singular vectors
 - Σ : diagonal
 - $r \times r$ diagonal matrix (r rank of matrix A)
 - Entries (called singular values) are positive, and sorted in decreasing order ($\sigma_1 \geq \sigma_2 \geq \dots \geq 0$)
- Remark: The decomposition is (almost) unique
 - see e.g. multiplication by -1

Singular Value Decomposition

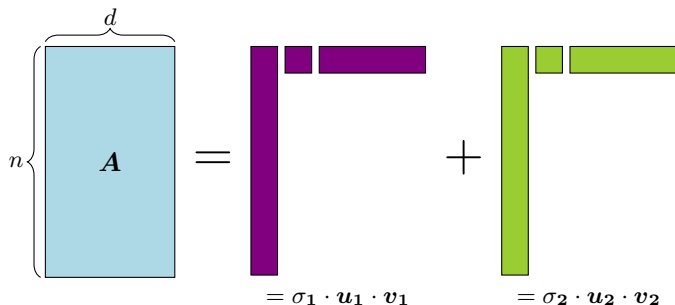
$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T = \sum_{i=1}^r \sigma_i \cdot \mathbf{u}_i \cdot \mathbf{v}_i^T \quad (7)$$



here: $r = 2$

Singular Value Decomposition

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T = \sum_{i=1}^r \sigma_i \cdot \mathbf{u}_i \cdot \mathbf{v}_i^T$$



here: $r = 2$

SVD Example: Users-to-Movies

- $A = U\Sigma V^T$ example: Users to Movies

$$\begin{array}{c} \text{Users} \end{array}
 \begin{array}{c} \text{Matrix} \\ \text{Alien} \\ \text{Serenity} \\ \text{Casablanca} \\ \text{Amelie} \end{array}
 \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 3 & 3 & 3 & 0 & 0 \\ 4 & 4 & 4 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 \\ 0 & 0 & 0 & 4 & 4 \\ 0 & 0 & 0 & 5 & 5 \\ 0 & 0 & 0 & 2 & 2 \end{bmatrix}
 =
 \begin{array}{c} \text{SciFi-concept} \\ \text{Romance-concept} \end{array}
 \begin{bmatrix} 0.14 & 0 \\ 0.42 & 0 \\ 0.56 & 0 \\ 0.70 & 0 \\ 0 & 0.59 \\ 0 & 0.74 \\ 0 & 0.29 \end{bmatrix}
 \times
 \begin{bmatrix} 12.36 & 0 \\ 0 & 9.48 \end{bmatrix}
 \times
 \begin{bmatrix} 0.57 & 0.57 & 0.57 & 0 & 0 \\ 0 & 0 & 0 & 0.70 & 0.70 \end{bmatrix}$$

SVD Example: Users-to-Movies

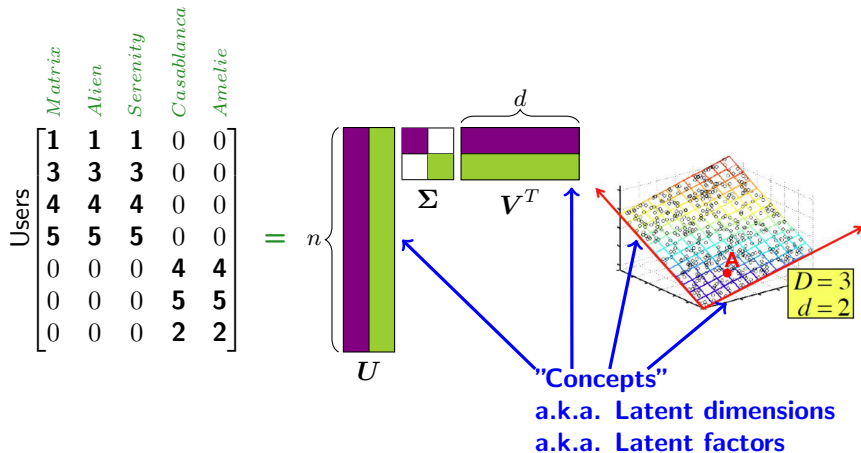
- $A = U\Sigma V^T$ example: Users to Movies

$$\begin{array}{c} \text{Users} \end{array}
 \begin{array}{c} \text{Matrix} \\ \text{Alien} \\ \text{Serenity} \\ \text{Casablanca} \\ \text{Amelie} \end{array}
 \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 3 & 3 & 3 & 0 & 0 \\ 4 & 4 & 4 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 \\ 0 & 0 & 0 & 4 & 4 \\ 0 & 0 & 0 & 5 & 5 \\ 0 & 0 & 0 & 2 & 2 \end{bmatrix}
 =
 \begin{array}{c} \text{SciFi-concept} \\ \downarrow \end{array}
 \begin{array}{c} \text{Romance-concept} \\ \swarrow \end{array}
 \begin{bmatrix} 0.14 & 0 \\ 0.42 & 0 \\ 0.56 & 0 \\ 0.70 & 0 \\ 0 & -0.59 \\ 0 & -0.74 \\ 0 & -0.29 \end{bmatrix}
 \times
 \begin{bmatrix} 12.36 & 0 \\ 0 & 9.48 \end{bmatrix}
 \times
 \begin{bmatrix} 0.57 & 0.57 & 0.57 & 0 & 0 \\ 0 & 0 & 0 & -0.70 & -0.70 \end{bmatrix}$$

(note the multiplication by -1)

SVD Example: Latent Factors

- $A = U\Sigma V^T$ example: Users to Movies



SVD Example: Beyond Blocks

$$\begin{array}{c} \text{Matrix} \\ \text{Alien} \\ \text{Serenity} \\ \text{Casablanca} \\ \text{Amelie} \end{array} \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 3 & 3 & 3 & 0 & 0 \\ 4 & 4 & 4 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 \\ 0 & \color{red}{2} & 0 & 4 & 4 \\ 0 & 0 & 0 & 5 & 5 \\ 0 & \color{red}{1} & 0 & 2 & 2 \end{bmatrix} = \begin{bmatrix} \mathbf{0.13} & -0.02 & -0.01 \\ \mathbf{0.41} & -0.07 & -0.03 \\ \mathbf{0.55} & -0.09 & -0.04 \\ \mathbf{0.68} & -0.11 & -0.05 \\ 0.15 & \mathbf{0.59} & \mathbf{0.65} \\ 0.07 & \mathbf{0.73} & \mathbf{-0.67} \\ 0.07 & \mathbf{0.29} & \mathbf{0.32} \end{bmatrix} \times \begin{bmatrix} \mathbf{12.4} & 0 & 0 \\ 0 & \mathbf{9.5} & 0 \\ 0 & 0 & \mathbf{1.3} \end{bmatrix} \times \begin{bmatrix} \mathbf{0.56} & \mathbf{0.59} & \mathbf{0.56} & 0.09 & 0.09 \\ -0.12 & 0.02 & -0.12 & \mathbf{0.69} & \mathbf{0.69} \\ 0.42 & \mathbf{-0.80} & 0.40 & 0.09 & 0.09 \end{bmatrix}$$

SVD Example: Beyond Blocks

$$\begin{array}{c} \text{Matrix} \\ \text{Alien} \\ \text{Serenity} \\ \text{Casablanca} \\ \text{Amelie} \end{array} \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 3 & 3 & 3 & 0 & 0 \\ 4 & 4 & 4 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 \\ 0 & 2 & 0 & 4 & 4 \\ 0 & 0 & 0 & 5 & 5 \\ 0 & 1 & 0 & 2 & 2 \end{bmatrix} = \begin{array}{c} \text{SciFi-concept} \\ \text{Romance-concept} \end{array} \begin{bmatrix} \mathbf{0.13} & -0.02 & -0.01 \\ \mathbf{0.41} & -0.07 & -0.03 \\ \mathbf{0.55} & -0.09 & -0.04 \\ \mathbf{0.68} & -0.11 & -0.05 \\ 0.15 & \mathbf{0.59} & \mathbf{0.65} \\ 0.07 & \mathbf{0.73} & \mathbf{-0.67} \\ 0.07 & \mathbf{0.29} & \mathbf{0.32} \end{bmatrix} \times \begin{bmatrix} \mathbf{12.4} & 0 & 0 \\ 0 & \mathbf{9.5} & 0 \\ 0 & 0 & \mathbf{1.3} \end{bmatrix} \times \begin{bmatrix} \mathbf{0.56} & \mathbf{0.59} & \mathbf{0.56} & 0.09 & 0.09 \\ -0.12 & 0.02 & -0.12 & \mathbf{0.69} & \mathbf{0.69} \\ 0.42 & \mathbf{-0.80} & 0.40 & 0.09 & 0.09 \end{bmatrix}$$

SVD Example: Beyond Blocks

U is "user-to-concept"
similarity matrix

$$\begin{array}{c} \text{Matrix} \\ \text{Alien} \\ \text{Serenity} \\ \text{Casablanca} \\ \text{Amelie} \end{array} \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 3 & 3 & 3 & 0 & 0 \\ 4 & 4 & 4 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 \\ 0 & 2 & 0 & 4 & 4 \\ 0 & 0 & 0 & 5 & 5 \\ 0 & 1 & 0 & 2 & 2 \end{bmatrix} = \begin{array}{c} \text{SciFi-concept} \\ \text{Romance-concept} \end{array} \begin{bmatrix} \mathbf{0.13} & -0.02 & -0.01 \\ \mathbf{0.41} & -0.07 & -0.03 \\ \mathbf{0.55} & -0.09 & -0.04 \\ \mathbf{0.68} & -0.11 & -0.05 \\ 0.15 & \mathbf{0.59} & \mathbf{0.65} \\ 0.07 & \mathbf{0.73} & \mathbf{-0.67} \\ 0.07 & \mathbf{0.29} & \mathbf{0.32} \end{bmatrix} \times \begin{bmatrix} \mathbf{12.4} & 0 & 0 \\ 0 & \mathbf{9.5} & 0 \\ 0 & 0 & \mathbf{1.3} \end{bmatrix} \times$$

$$\begin{bmatrix} \mathbf{0.56} & \mathbf{0.59} & \mathbf{0.56} & 0.09 & 0.09 \\ -0.12 & 0.02 & -0.12 & \mathbf{0.69} & \mathbf{0.69} \\ 0.42 & \mathbf{-0.80} & 0.40 & 0.09 & 0.09 \end{bmatrix}$$

SVD Example: Beyond Blocks

$$\begin{array}{c} \text{Matrix} \\ \text{Alien} \\ \text{Serenity} \\ \text{Casablanca} \\ \text{Amelie} \end{array} \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 3 & 3 & 3 & 0 & 0 \\ 4 & 4 & 4 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 \\ 0 & 2 & 0 & 4 & 4 \\ 0 & 0 & 0 & 5 & 5 \\ 0 & 1 & 0 & 2 & 2 \end{bmatrix} = \begin{array}{c} \text{SciFi-concept} \\ \downarrow \end{array} \begin{bmatrix} \mathbf{0.13} & -0.02 & -0.01 \\ \mathbf{0.41} & -0.07 & -0.03 \\ \mathbf{0.55} & -0.09 & -0.04 \\ \mathbf{0.68} & -0.11 & -0.05 \\ 0.15 & \mathbf{0.59} & \mathbf{0.65} \\ 0.07 & \mathbf{0.73} & \mathbf{-0.67} \\ 0.07 & \mathbf{0.29} & \mathbf{0.32} \end{bmatrix} \times \begin{array}{c} \text{"strength" of the SciFi-concept} \\ \downarrow \end{array} \begin{bmatrix} \mathbf{12.4} & 0 & 0 \\ 0 & \mathbf{9.5} & 0 \\ 0 & 0 & \mathbf{1.3} \end{bmatrix} \times$$

$$\begin{bmatrix} \mathbf{0.56} & \mathbf{0.59} & \mathbf{0.56} & 0.09 & 0.09 \\ -0.12 & 0.02 & -0.12 & \mathbf{0.69} & \mathbf{0.69} \\ 0.42 & \mathbf{-0.80} & 0.40 & 0.09 & 0.09 \end{bmatrix}$$

SVD Example: Beyond Blocks

V is "movie-to-concept" similarity matrix

$$\begin{array}{c} \text{Matrix} \\ \text{Alien} \\ \text{Serenity} \\ \text{Casablanca} \\ \text{Amelie} \end{array} \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 3 & 3 & 3 & 0 & 0 \\ 4 & 4 & 4 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 \\ 0 & 2 & 0 & 4 & 4 \\ 0 & 0 & 0 & 5 & 5 \\ 0 & 1 & 0 & 2 & 2 \end{bmatrix} = \begin{array}{c} \text{SciFi-concept} \\ \downarrow \\ \begin{bmatrix} 0.13 & -0.02 & -0.01 \\ 0.41 & -0.07 & -0.03 \\ 0.55 & -0.09 & -0.04 \\ 0.68 & -0.11 & -0.05 \\ 0.15 & 0.59 & 0.65 \\ 0.07 & 0.73 & -0.67 \\ 0.07 & 0.29 & 0.32 \end{bmatrix} \end{array} \times \begin{bmatrix} 12.4 & 0 & 0 \\ 0 & 9.5 & 0 \\ 0 & 0 & 1.3 \end{bmatrix} \times$$

$$\begin{array}{c} \text{SciFi-concept} \\ \uparrow \\ \begin{bmatrix} 0.56 & 0.59 & 0.56 & 0.09 & 0.09 \\ -0.12 & 0.02 & -0.12 & 0.69 & 0.69 \\ 0.42 & -0.80 & 0.40 & 0.09 & 0.09 \end{bmatrix} \end{array}$$

SVD: Interpretation

- $A = U\Sigma V^T$
- 'movies', 'users' and 'concepts':
 - A : original data: movies-to-users
 - U : user-to-concept similarity matrix
 - V : movie-to-concept similarity matrix
 - Σ : its diagonal elements: 'strength' of each concept
- **Benefits of SVD (or in general matrix decomposition):**
 - Discover hidden correlations/topics
 - Words that occur commonly together; movies of the same genre; ...
 - Interpretation and visualization

Chapter: Dimensionality Reduction & Matrix Factorization

1. Introduction
2. Principal Component Analysis (PCA)
3. **Singular Value Decomposition (SVD)**
 - Idea: Low Rank Approximation
 - SVD & Latent Factors
 - **Dimensionality Reduction**
4. Matrix Factorization
5. Neighbor Graph Methods
6. Autoencoders (Nonlinear Dimensionality Reduction)

Recap: Dim. Reduction by Low Rank Approx.

- Idea: approximate original data $\mathbf{A}$ by a low rank matrix $\mathbf{B}$

$$\mathbf{A} = \begin{bmatrix} 1.01 & 2.05 & 0.9 \\ -2.1 & -3.05 & 1.1 \\ 2.99 & 5.031 & 0.3 \end{bmatrix} \approx \mathbf{B} = \begin{bmatrix} 1 & 2 & 1 \\ -2 & -3 & 1 \\ 3 & 5 & 0 \end{bmatrix}$$

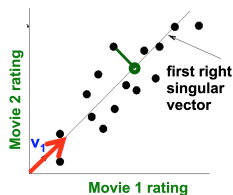
- Important: Even though both $\mathbf{A}$ and $\mathbf{B}$ are $\in \mathbb{R}^{n \times d}$ we need only two coordinates per point to describe $\mathbf{B}$
 - $\text{rank}(\mathbf{A}) = 3$ vs. $\text{rank}(\mathbf{B}) = 2$ (3 vs. 2 coordinates per point)
- Goal: Find the best low-rank approximation**
 - best = minimize the sum of reconstruction error
 - Given matrix $\mathbf{A} \in \mathbb{R}^{n \times d}$, find $\mathbf{B} \in \mathbb{R}^{n \times d}$ with $\text{rank}(\mathbf{B}) = k$ that minimizes

$$\|\mathbf{A} - \mathbf{B}\|_F^2 = \sum_{i=1}^N \sum_{j=1}^D (a_{ij} - b_{ij})^2 \quad (8)$$

SVD: Alternative Interpretation

- $A = U\Sigma V^T$ - example:

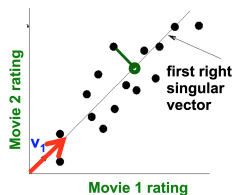
Variance ("spread")
on the v_1 axis



$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 3 & 3 & 3 & 0 & 0 \\ 4 & 4 & 4 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 \\ 0 & 2 & 0 & 4 & 4 \\ 0 & 0 & 0 & 5 & 5 \\ 0 & 1 & 0 & 2 & 2 \end{bmatrix} = \begin{bmatrix} 0.13 & -0.02 & -0.01 \\ 0.41 & -0.07 & -0.03 \\ 0.55 & -0.09 & -0.04 \\ 0.68 & -0.11 & -0.05 \\ 0.15 & 0.59 & 0.65 \\ 0.07 & 0.73 & -0.67 \\ 0.07 & 0.29 & 0.32 \end{bmatrix} \times \begin{bmatrix} 12.4 & 0 & 0 \\ 0 & 9.5 & 0 \\ 0 & 0 & 1.3 \end{bmatrix} \times \begin{bmatrix} 0.56 & 0.59 & 0.56 & 0.09 & 0.09 \\ -0.12 & 0.02 & -0.12 & 0.69 & 0.69 \\ 0.42 & -0.80 & 0.40 & 0.09 & 0.09 \end{bmatrix}$$

SVD: Alternative Interpretation

- $A = U\Sigma V^T$
- $U\Sigma$: Gives the coordinates of the points in the projection axis



$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 3 & 3 & 3 & 0 & 0 \\ 4 & 4 & 4 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 \\ 0 & 2 & 0 & 4 & 4 \\ 0 & 0 & 0 & 5 & 5 \\ 0 & 1 & 0 & 2 & 2 \end{bmatrix}$$

Projection of users on
the "Sci-Fi" axis $(U\Sigma)^T$

$$\begin{bmatrix} 1.61 & 0.19 & -0.01 \\ 5.08 & 0.66 & -0.03 \\ 6.82 & 0.85 & -0.05 \\ 8.43 & 1.04 & -0.06 \\ 1.86 & -5.60 & 0.84 \\ 0.86 & -6.93 & -0.87 \\ 0.86 & -2.75 & 0.41 \end{bmatrix}$$

SVD: Best Approximation

- How to find the best approximation?

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 3 & 3 & 3 & 0 & 0 \\ 4 & 4 & 4 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 \\ 0 & 2 & 0 & 4 & 4 \\ 0 & 0 & 0 & 5 & 5 \\ 0 & 1 & 0 & 2 & 2 \end{bmatrix} = \begin{bmatrix} \mathbf{0.13} & -0.02 & -0.01 \\ \mathbf{0.41} & -0.07 & -0.03 \\ \mathbf{0.55} & -0.09 & -0.04 \\ \mathbf{0.68} & -0.11 & -0.05 \\ 0.15 & \mathbf{0.59} & \mathbf{0.65} \\ 0.07 & \mathbf{0.73} & \mathbf{-0.67} \\ 0.07 & \mathbf{0.29} & \mathbf{0.32} \end{bmatrix} \times \begin{bmatrix} \mathbf{12.4} & 0 & 0 \\ 0 & \mathbf{9.5} & 0 \\ 0 & 0 & \mathbf{1.3} \end{bmatrix} \times \begin{bmatrix} \mathbf{0.56} & \mathbf{0.59} & \mathbf{0.56} & 0.09 & 0.09 \\ -0.12 & 0.02 & -0.12 & \mathbf{0.69} & \mathbf{0.69} \\ 0.42 & \mathbf{-0.80} & 0.40 & 0.09 & 0.09 \end{bmatrix}$$

SVD: Best Approximation

- How to find the best approximation?
- Set smallest singular values to zero!

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 3 & 3 & 3 & 0 & 0 \\ 4 & 4 & 4 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 \\ 0 & 2 & 0 & 4 & 4 \\ 0 & 0 & 0 & 5 & 5 \\ 0 & 1 & 0 & 2 & 2 \end{bmatrix} = \begin{bmatrix} \mathbf{0.13} & -0.02 & -0.01 \\ \mathbf{0.41} & -0.07 & -0.03 \\ \mathbf{0.55} & -0.09 & -0.04 \\ \mathbf{0.68} & -0.11 & -0.05 \\ 0.15 & \mathbf{0.59} & \mathbf{0.65} \\ 0.07 & \mathbf{0.73} & \mathbf{-0.67} \\ 0.07 & \mathbf{0.29} & \mathbf{0.32} \end{bmatrix} \times \begin{bmatrix} \mathbf{12.4} & 0 & 0 \\ 0 & \mathbf{9.5} & 0 \\ 0 & 0 & \del{1.3} \end{bmatrix} \times \begin{bmatrix} \mathbf{0.56} & \mathbf{0.59} & \mathbf{0.56} & 0.09 & 0.09 \\ -0.12 & 0.02 & -0.12 & \mathbf{0.69} & \mathbf{0.69} \\ 0.42 & \mathbf{-0.80} & 0.40 & 0.09 & 0.09 \end{bmatrix}$$

SVD: Best Approximation

- How to find the best approximation?
- Set smallest singular values to zero!

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 3 & 3 & 3 & 0 & 0 \\ 4 & 4 & 4 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 \\ 0 & 2 & 0 & 4 & 4 \\ 0 & 0 & 0 & 5 & 5 \\ 0 & 1 & 0 & 2 & 2 \end{bmatrix} \approx \begin{bmatrix} \mathbf{0.13} & -0.02 & -0.01 \\ \mathbf{0.41} & -0.07 & -0.03 \\ \mathbf{0.55} & -0.09 & -0.04 \\ \mathbf{0.68} & -0.11 & -0.05 \\ 0.15 & \mathbf{0.59} & \mathbf{0.65} \\ 0.07 & \mathbf{0.73} & -\mathbf{0.67} \\ 0.07 & \mathbf{0.29} & \mathbf{0.32} \end{bmatrix} \times \begin{bmatrix} \mathbf{12.4} & 0 & 0 \\ 0 & \mathbf{9.5} & 0 \\ 0 & 0 & \mathbf{1.3} \end{bmatrix} \times \begin{bmatrix} \mathbf{0.56} & \mathbf{0.59} & \mathbf{0.56} & 0.09 & 0.09 \\ -0.12 & 0.02 & -0.12 & \mathbf{0.69} & \mathbf{0.69} \\ \mathbf{0.42} & \mathbf{-0.80} & \mathbf{0.40} & \mathbf{0.09} & \mathbf{0.09} \end{bmatrix}$$

SVD: Best Approximation

- How to find the best approximation?
- Set smallest singular values to zero!

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 3 & 3 & 3 & 0 & 0 \\ 4 & 4 & 4 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 \\ 0 & 2 & 0 & 4 & 4 \\ 0 & 0 & 0 & 5 & 5 \\ 0 & 1 & 0 & 2 & 2 \end{bmatrix} \approx \begin{bmatrix} \mathbf{0.13} & -0.02 \\ \mathbf{0.41} & -0.07 \\ \mathbf{0.55} & -0.09 \\ \mathbf{0.68} & -0.11 \\ 0.15 & \mathbf{0.59} \\ 0.07 & \mathbf{0.73} \\ 0.07 & \mathbf{0.29} \end{bmatrix} \times \begin{bmatrix} \mathbf{12.4} & 0 \\ 0 & \mathbf{9.5} \end{bmatrix} \times \begin{bmatrix} \mathbf{0.56} & \mathbf{0.59} & \mathbf{0.56} & 0.09 & 0.09 \\ -0.12 & 0.02 & -0.12 & \mathbf{0.69} & \mathbf{0.69} \end{bmatrix}$$

SVD: Best Approximation

- How to find the best approximation?
- Set smallest singular values to zero!

By construction,
the rank of the
new matrix is 2

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 3 & 3 & 3 & 0 & 0 \\ 4 & 4 & 4 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 \\ 0 & 2 & 0 & 4 & 4 \\ 0 & 0 & 0 & 5 & 5 \\ 0 & 1 & 0 & 2 & 2 \end{bmatrix} \approx \begin{bmatrix} \mathbf{0.92} & \mathbf{0.95} & \mathbf{0.92} & 0.01 & 0.01 \\ \mathbf{2.91} & \mathbf{3.01} & \mathbf{2.91} & -0.01 & -0.01 \\ \mathbf{3.90} & \mathbf{4.04} & \mathbf{3.90} & 0.01 & 0.01 \\ \mathbf{4.82} & \mathbf{5.00} & \mathbf{4.82} & 0.03 & 0.03 \\ 0.70 & \mathbf{0.53} & 0.70 & \mathbf{4.11} & \mathbf{4.11} \\ -0.69 & 1.34 & -0.69 & \mathbf{4.78} & \mathbf{4.78} \\ 0.32 & \mathbf{0.23} & 0.32 & \mathbf{2.01} & \mathbf{2.01} \end{bmatrix}$$

SVD: Best Low Rank Approximation

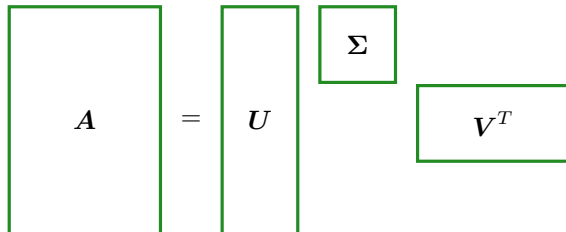


Diagram illustrating the SVD decomposition of matrix A :

$$A = U \Sigma V^T$$

The matrix A is represented by a large vertical rectangle. It is equal to the product of matrix U (a tall, narrow vertical rectangle), matrix Σ (a small square), and matrix V^T (a wide, short horizontal rectangle).

B is best approximation of A

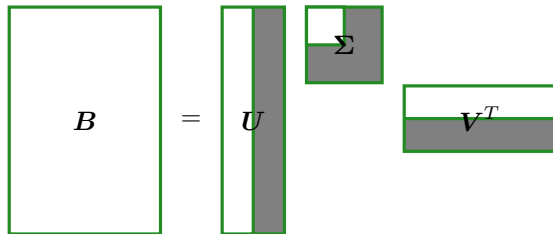


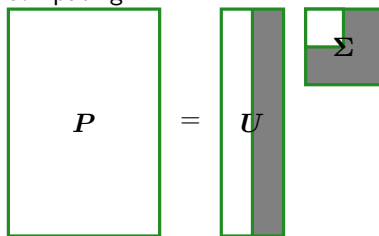
Diagram illustrating the truncated SVD decomposition, where B is the best low-rank approximation of A :

$$B = U \Sigma V^T$$

The matrix B is represented by a large vertical rectangle. It is equal to the product of matrix U (a tall, narrow vertical rectangle with a grey vertical band), matrix Σ (a small square with a grey bottom-right portion), and matrix V^T (a wide, short horizontal rectangle with a grey bottom portion).

SVD: Projection

- Note: The actual projected/reduced data can be obtained by computing


$$P = U \Sigma V^T$$

- Or equivalently: $P = A \cdot V$ (since V is orthonormal)

Best Approximation – Intuitive Explanation

- Recap: Vectors u_i and v_i are of unit length
- W.l.o.g.: $\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq \dots \geq 0$

$$\begin{bmatrix} \mathbf{1} & \mathbf{1} & \mathbf{1} & 0 & 0 \\ \mathbf{3} & \mathbf{3} & \mathbf{3} & 0 & 0 \\ \mathbf{4} & \mathbf{4} & \mathbf{4} & 0 & 0 \\ \mathbf{5} & \mathbf{5} & \mathbf{5} & 0 & 0 \\ 0 & \mathbf{2} & 0 & \mathbf{4} & \mathbf{4} \\ 0 & 0 & 0 & \mathbf{5} & \mathbf{5} \\ 0 & \mathbf{1} & 0 & \mathbf{2} & \mathbf{2} \end{bmatrix} = \begin{bmatrix} | & | \\ \mathbf{u}_1 & \mathbf{u}_2 \\ | & | \end{bmatrix} \mathbf{x} \begin{bmatrix} \sigma_1 & \emptyset \\ \emptyset & \sigma_2 \end{bmatrix} \mathbf{x} \begin{bmatrix} - & \mathbf{v}_1 & - \\ - & \mathbf{v}_2 & - \end{bmatrix}$$

Best Approximation – Intuitive Explanation

- Recap: Vectors $\mathbf{u}_i$ and $\mathbf{v}_i$ are of unit length
- W.l.o.g.: $\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq \dots \geq 0$

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 3 & 3 & 3 & 0 & 0 \\ 4 & 4 & 4 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 \\ 0 & 2 & 0 & 4 & 4 \\ 0 & 0 & 0 & 5 & 5 \\ 0 & 1 & 0 & 2 & 2 \end{bmatrix} = \overbrace{\sigma_1 \mathbf{u}_1 \mathbf{v}_1^T + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^T + \dots}^{r \text{ terms}}$$

Q: How many σ_i to pick?

A: Rule of thumb:

keep 90% of 'energy'

$$\sum_{i=1}^k \sigma_i^2 = 0.9 \sum_{i=1}^r \sigma_i^2$$

- σ_i scales the terms $\mathbf{u}_i \cdot \mathbf{v}_i^T$
- Zeroing small σ_i introduces less error

SVD: Best Low Rank Approximation – Proof

- Theorem: Let $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ ($\sigma_1 \geq \sigma_2 \geq \dots \geq 0, \text{rank}(\mathbf{A}) = r$) and $\mathbf{B} = \mathbf{U}\mathbf{S}\mathbf{V}^T$ with $\mathbf{S}$ being a diagonal $r \times r$ matrix where

- $s_i = \sigma_i$ for $i = 1, \dots, k$ and $s_i = 0$ for $i = k + 1, \dots, d$

Then $\mathbf{B}$ is a best rank= k approximation to $\mathbf{A}$ regarding Frobenius norm, i.e. $\mathbf{B}$ is a solution to $\min_{\mathbf{B}} \|\mathbf{A} - \mathbf{B}\|_F$ where $\text{rank}(\mathbf{B}) = k$

- We have uploaded a detailed proof to the web
 - Note: Many proofs on the web and on other lecture slides are incorrect!
- Remark: $\mathbf{B}$ is also an optimal low-rank approximation regarding the spectral norm (operator 2-norm): $\min_{\mathbf{B}} \|\mathbf{A} - \mathbf{B}\|_2$
 - $\|\mathbf{X}\|_2 = \text{largest singular value of } \mathbf{X}$

SVD: Best Low Rank Approximation – Proof

Some facts:

- $\|\mathbf{X}\|_F = \|\mathbf{X}^T\|_F$
 - obvious from the definition
- $\|\mathbf{X}\|_F^2 = \text{trace}(\mathbf{X}^T \mathbf{X})$
- Frobeniusnorm is invariant to orthonormal transformations $\mathbf{U}$
 - Note: if $\mathbf{U}^T \mathbf{U} = \mathbf{I}$ then also $\mathbf{U} \mathbf{U}^T = \mathbf{I}$
 - $\|\mathbf{U} \mathbf{X}\|_F^2 = \text{trace}((\mathbf{U} \mathbf{X})^T (\mathbf{U} \mathbf{X})) = \text{trace}(\mathbf{X}^T \mathbf{U}^T \mathbf{U} \mathbf{X}) = \text{trace}(\mathbf{X}^T \mathbf{X}) = \|\mathbf{X}\|_F^2$
 - $\|\mathbf{X} \mathbf{U}\|_F^2 = \|(\mathbf{X} \mathbf{U})^T\|_F^2 = \text{trace}((\mathbf{X} \mathbf{U})(\mathbf{X} \mathbf{U})^T) = \text{trace}(\mathbf{X} \mathbf{U} \mathbf{U}^T \mathbf{X}^T) = \text{trace}(\mathbf{X} \mathbf{X}^T) = \|\mathbf{X}\|_F^2$
- Let $\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$ then $\|\mathbf{A}\|_F^2 = \|\mathbf{\Sigma}\|_F^2 = \sum_i^r \sigma_i^2$
 - follows from above results

SVD: Complexity

- To compute SVD:
 - $O(n \cdot d^2)$ or $O(n^2 \cdot d)$ (whichever is smaller)
- But:
 - Less work, if we just want singular values
 - or if we want first k singular vectors
 - or if the matrix is sparse
- Implemented in linear algebra packages like:
 - LINPACK, Matlab, SPlus, Mathematica . . .

SVD & PCA: Comparison

- Given data $\mathbf{X}$ (let's assume it is already centered)
- SVD gives us
 - $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$
 - Projected data obtained by $\mathbf{X}\mathbf{V} = \mathbf{U}\mathbf{\Sigma}$ (or truncated $\mathbf{V}$)
- PCA computes the eigendecomposition of the covariance matrix
 - Covariance matrix: $\mathbf{X}^T\mathbf{X}$
 - Eigendecomposition leads to $\mathbf{X}^T\mathbf{X} = \mathbf{\Gamma} \cdot \mathbf{\Lambda} \cdot \mathbf{\Gamma}^T$
 - Projected data obtained by $\mathbf{X} \cdot \mathbf{\Gamma}$ (or truncated $\mathbf{\Gamma}$)
- Let us calculate:
 - $\mathbf{X}^T\mathbf{X} = (\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T)^T\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T = \mathbf{V}\mathbf{\Sigma}^T\mathbf{U}^T(\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T) = \mathbf{V}\mathbf{\Sigma}\mathbf{\Sigma}^T\mathbf{V}^T = \mathbf{V}\mathbf{\Sigma}^2\mathbf{V}^T$
 - $\mathbf{\Gamma} = \mathbf{V}$
 - $\mathbf{\Sigma}^2 = \mathbf{\Lambda}$

SVD & PCA: Comparison

- Given data $\mathbf{X}$ (let's assume it is already centered)
- SVD gives us
 - $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$
 - Projected data obtained by $\mathbf{X}\mathbf{V} = \mathbf{U}\mathbf{\Sigma}$ (or truncated $\mathbf{V}$)
- PCA computes the eigendecomposition of the covariance matrix
 - Covariance matrix: $\mathbf{X}^T\mathbf{X}$
 - Eigendecomposition leads to $\mathbf{X}^T\mathbf{X} = \mathbf{\Gamma} \cdot \mathbf{\Lambda} \cdot \mathbf{\Gamma}^T$
 - Projected data obtained by $\mathbf{X} \cdot \mathbf{\Gamma}$ (or truncated $\mathbf{\Gamma}$)
- Let us calculate:
 - $\mathbf{X}^T\mathbf{X} = (\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T)^T\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T = \mathbf{V}\mathbf{\Sigma}^T\mathbf{U}^T(\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T) = \mathbf{V}\mathbf{\Sigma}\mathbf{\Sigma}^T\mathbf{V}^T = \mathbf{V}\mathbf{\Sigma}^2\mathbf{V}^T$
 - $\mathbf{\Gamma} = \mathbf{V}$
 - $\mathbf{\Sigma}^2 = \mathbf{\Lambda}$

PCA and SVD are equivalent!

SVD & PCA: Comparison

- Given data $\mathbf{X}$ (let's assume it is already centered)
 - SVD gives us
 - $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$
 - Projected data obtained by $\mathbf{X}\mathbf{V} = \mathbf{U}\mathbf{\Sigma}$ (or truncated $\mathbf{V}$)
 - PCA computes the eigendecomposition of the covariance matrix
 - Covariance matrix: $\mathbf{X}^T\mathbf{X}$
 - Eigendecomposition leads to $\mathbf{X}^T\mathbf{X} = \mathbf{\Gamma} \cdot \mathbf{\Lambda} \cdot \mathbf{\Gamma}^T$
 - Projected data obtained by $\mathbf{X} \cdot \mathbf{\Gamma}$ (or truncated $\mathbf{\Gamma}$)
 - Let us calculate:
 - $\mathbf{X}^T\mathbf{X} = (\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T)^T\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T = \mathbf{V}\mathbf{\Sigma}^T\mathbf{U}^T(\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T) = \mathbf{V}\mathbf{\Sigma}\mathbf{\Sigma}^T\mathbf{V}^T = \mathbf{V}\mathbf{\Sigma}^2\mathbf{V}^T$
 - $\mathbf{\Gamma} = \mathbf{V}$
 - $\mathbf{\Sigma}^2 = \mathbf{\Lambda}$
- PCA and SVD are equivalent!**
- Squared singular values are variances in new space!**

SVD & PCA: Comparison

Transform the data such that dimensions of new space are uncorrelated
+ discard (new) dimensions with smallest variance

=

find optimal low-rank approximation
(regarding Frobeniusnorm)

Remark: Computation of SVD

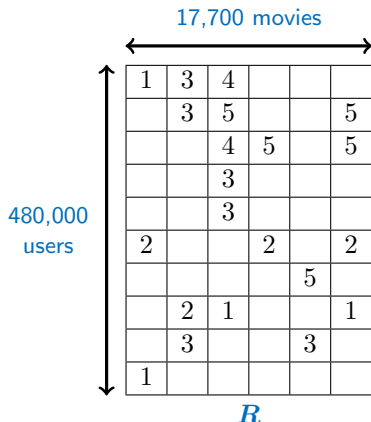
- We can use the eigendecomposition to calculate the singular value decomposition
- $X^T X = (U \Sigma V^T)^T U \Sigma V^T = V \Sigma^T U^T (U \Sigma V^T) = V \Sigma \Sigma^T V^T = V \Sigma^2 V^T$
 - V = eigenvectors of $X^T X$
- $XX^T = U \Sigma V^T (V \Sigma^T U^T) = U \Sigma \Sigma^T U^T$
 - U = eigenvectors of XX^T
- Drawback: Numerically unstable
 - Better to use specialized algorithms

Chapter: Dimensionality Reduction & Matrix Factorization

1. Introduction
2. Principal Component Analysis (PCA)
3. Singular Value Decomposition (SVD)
4. **Matrix Factorization**
 - **Motivation & Approach**
 - Regularization & Sparsity
 - Further Factorization Models
5. Neighbor Graph Methods
6. Autoencoders (Nonlinear Dimensionality Reduction)

Motivation: The Netflix Prize

- Training data
 - 100 million ratings, 480,000 users, 17,770 movies
 - 6 years of data: 2000-2005
- Test data
 - Last few ratings of each user (2.8 million)
 - Root Mean Square Error (RMSE)
 - Netflix's system RMSE: 0.9514
- Competition
 - 2,700+ teams
 - \$1 million prize for 10% improvement



Evaluating Recommender Systems

- S = set of tuples (u, i) of users u that have rated item i with a rating of r_{ui}

- $$RMSE = \sqrt{\frac{1}{|S|} \sum_{(u,i) \in S} (r_{ui} - \hat{r}_{ui})^2}$$

true rating of
user u for item i

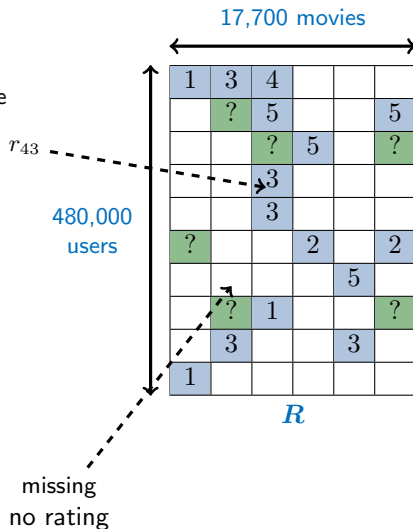
r_{ui}

predicted
rating

$\hat{r}_{ui}$

- Legend:

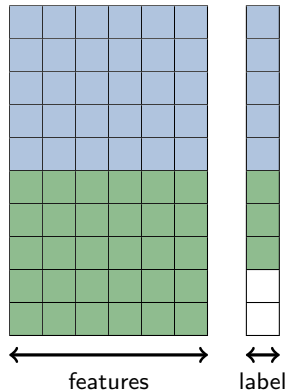
- Training and validation data
- Test data
- Missing data



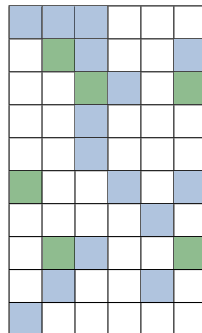
Regression vs. Recommendation

- Legend:
 - Training and validation data
 - Test data
 - Missing data

Regression



Recommendation



SVD on Rating Data

- Goal: Make good recommendations
 - Good performance on observed (user, item) ratings, i.e. low RMSE
 - Generalize to the unseen test data
- Can we use SVD to obtain the solution?
 - SVD on the rating matrix $R \in \mathbb{R}^{n \times d}$ where we replace **missing** entries with **zeros**
 - $R \approx Q \cdot P^T$ // SVD: $R = U \Sigma V^T \rightarrow Q = U \Sigma, P = V$

	Items													Factors			
Users	1	0	3	0	0	5	0	0	5	0	4	0	$\approx$	.1	-.4	.2	$\times$
	0	0	5	4	0	0	4	0	0	2	1	3		-.5	.6	.5	
	2	4	0	1	2	0	3	0	4	3	5	0		-.2	.3	.5	
	0	2	4	0	5	0	0	4	0	0	2	0		1.1	2.1	.3	
	0	0	4	3	4	2	0	0	0	0	2	5		-.7	2.1	-.2	
	1	0	3	0	3	0	0	2	0	0	4	0		-1	.7	.3	
	R													Q			

SVD on Rating Data

- SVD gives **minimum reconstruction error** (Sum of Squared Errors):

$$\min_{U, \Sigma, V} \sum_{\substack{u=1 \dots n \\ i=1 \dots d}} \left(R_{ui} - [U \Sigma V^T]_{ui} \right)^2$$

- SSE and RMSE are monotonically related:
 - $RMSE = \frac{1}{const.} \sqrt{SSE}$
 - Great news: SVD is minimizing RMSE
- **Complication:**
 - The sum in the SVD error term is over **all** entries (no rating = zero rating)
 - But our R has **missing** entries!
 - (Classical) SVD isn't defined when entries are missing!
- Also: We actually don't care about orthogonality and normalization

Discussion: SVD/PCA

- Optimal low-rank approximation
 - In terms of Frobenius norm (and also in terms of the spectral norm)
- Missing elements (we have no information/not observed) are treated as 0 (a very low rating)
 - Critical for many applications (e.g. recommender systems)
 - General problem: two kinds of "zeros" (not observed/missing $\neq$ 0)
- Lack of Sparsity
 - Singular vectors are dense
 - Potential interpretability problem
- Orthogonality really required/useful?

$$\begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & & \cdot \\ \cdot & & \\ \cdot & & \\ & \cdot & \end{bmatrix} = \begin{bmatrix} \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \end{bmatrix} \begin{bmatrix} \cdot & & & & \\ & \cdot & & & \\ & & \cdot & & \\ & & & \cdot & \\ & & & & \cdot \end{bmatrix} \begin{bmatrix} \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \end{bmatrix}^T$$

The diagram illustrates the SVD decomposition of a matrix. On the left is a sparse matrix represented by a rectangle containing several black dots. This is followed by an equals sign. To the right of the equals sign are three matrices: a tall, dense matrix labeled U (represented by a rectangle filled with black squares), a small square diagonal matrix labeled Σ (represented by a small rectangle with a few black squares on the diagonal), and a wide, dense matrix labeled V^T (represented by a rectangle filled with black squares).

Latent Factor Models

- Our goal: Find $\mathbf{Q} \in \mathbb{R}^{n \times k}$ and $\mathbf{P} \in \mathbb{R}^{d \times k}$ such that:

$$\min_{\mathbf{P}, \mathbf{Q}} \sum_{(u,i) \in S} (r_{ui} - \mathbf{q}_u \cdot \mathbf{p}_i^T)^2$$

- We only sum over existing entries, i.e. the set $S = \{(u,i) | r_{ui} \neq \text{missing}\}$
- We don't require columns of $\mathbf{P}, \mathbf{Q}$ to be orthogonal/ unit length
- Use standard optimization techniques to solve this problem

$$\hat{r}_{ui} = \mathbf{q}_u \cdot \mathbf{p}_i^T$$

$\mathbf{q}_u$ row u of $\mathbf{Q}$
 $\mathbf{p}_i$ row i of $\mathbf{P}$

	Items													Factors											
Users	1	0	3	0	0	5	0	0	5	0	4	0	$\approx$	Users	.1	-.4	.2	q_u row u of Q							
	0	0	5	4	0	0	4	0	0	2	1	3			-.5	.6	.5		p_i row i of P^T						
	2	4	0	1	2	0	3	0	4	3	5	0			-.2	.3	.5								
	0	2	4	0	5	0	0	4	0	0	2	0			1.1	2.1	.3								
	0	0	4	3	4	2	0	0	0	0	2	5			-.7	2.1	-.2								
	1	0	3	0	3	0	0	2	0	0	4	0			-1	.7	.3								
	R													Q											
Factors	Items												P^T												
	1.1	-.2	.3	.5	-.2	-.5	.8	-.4	.3	1.4	2.4	-.9													
	-.8	.7	.5	1.4	.3	-1	1.4	2.9	-.7	1.2	-.1	1.3													
	2.1	-.4	.6	1.7	2.4	.9	-.3	.4	.8	.7	-.6	.1													

Finding the Latent Factors (I)

- Goal: $\min_{\mathbf{P}, \mathbf{Q}} \sum_{(u,i) \in S} (r_{ui} - \mathbf{q}_u \cdot \mathbf{p}_i^T)^2 =: \min_{\mathbf{P}, \mathbf{Q}} f(\mathbf{P}, \mathbf{Q})$
- One approach: **Alternating Optimization** //a.k.a. block coordinate minimization
 - Pick initial values for $\mathbf{P}$ and $\mathbf{Q}$
 - Alternatingly keep one variable fix and optimize for the other
 - Repeat until convergence
- Pseudo Code:
 1. Initialize $\mathbf{P}^{(0)}, \mathbf{Q}^{(0)}, t = 0$
 2. $\mathbf{P}^{(t+1)} = \arg \min_{\mathbf{P}} f(\mathbf{P}, \mathbf{Q}^{(t)})$
 3. $\mathbf{Q}^{(t+1)} = \arg \min_{\mathbf{Q}} f(\mathbf{P}^{t+1}, \mathbf{Q})$
 4. $t = t + 1$
 5. goto 2 until convergence

Finding the Latent Factors (II)

- Initialization of $\mathbf{P}$ and $\mathbf{Q}$:

- Use, e.g., SVD where missing entries are replaced by 0 (or overall mean)

- How to solve

$$\mathbf{P}^{(t+1)} = \arg \min_{\mathbf{P}} f(\mathbf{P}, \mathbf{A}^{(t)}) = \arg \min_{\mathbf{P}} \sum_{(u,i) \in S} (r_{ui} - \mathbf{q}_u \cdot \mathbf{p}_i^T)^2$$

- Observation 1: Since $\mathbf{Q}$ is fixed, the problem can be solved for **each** vector $\mathbf{p}_i$ **independently**

$$\min_{\mathbf{P}} \sum_{(u,i) \in S} (r_{ui} - \mathbf{q}_u \cdot \mathbf{p}_i^T)^2 = \sum_{i=1 \dots d} \min_{\mathbf{p}_i} \sum_{u \in S_{*,i}} (r_{ui} - \mathbf{q}_u \cdot \mathbf{p}_i^T)^2$$

where $S_{*,i} = \{u | (u,i) \in S\}$ // = only users u who have rated item i

- Equivalently for vector $\mathbf{q}_u$ based on the set $S_{*,i} = \{u | (u,i) \in S\}$

Finding the Latent Factors (III)

- *Observation 2:* $\min_{\mathbf{p}_i} \sum_{u \in S_{*,i}} (r_{ui} - \mathbf{q}_u \cdot \mathbf{p}_i^T)^2$ is an **ordinary least squares regression** problem
 - $\min_{\mathbf{w}} \sum_{j=1}^n (y_j - \mathbf{w}^T \mathbf{x}_j)$ // y_j is a scalar, $\mathbf{x}_j$ and $\mathbf{w}$ (column) vectors
 - Optimal solution in closed form:
$$\mathbf{w} = \left(\frac{1}{n} \sum_{j=1}^n \mathbf{x}_j \mathbf{x}_j^T \right)^{-1} \cdot \left(\frac{1}{n} \sum_{j=1}^n \mathbf{x}_j y_j^T \right)^{-1}$$
 - In our case: $\mathbf{p}_i^T = \left(\frac{1}{|S_{*,i}|} \sum_{u \in S_{*,i}} \mathbf{q}_u^T \mathbf{q}_u \right)^{-1} \cdot \frac{1}{|S_{*,i}|} \sum_{u \in S_{*,i}} \mathbf{q}_u^T r_{ui}$
 - Equivalently for $\mathbf{q}_u$
- Computation of $\arg \min_{\mathbf{P}} (\mathbf{P}, \mathbf{Q}^{(t)})$ reduces to a standard problem

Alternating Optimization: Discussion

- May provide solution to difficult optimization problems
- Here: sequence of simple OLS problems
 - Overall algorithm can be implemented in a few lines in, e.g., Python
 - Quite efficient: since data is sparse, the sets $S_{*,i}$ (and $S_{u,*}$) are relatively small
- **Drawback** of Alternating Optimization:
 - Solution is only an approximation
 - No guarantee that close to the optimal solution
 - Highly depends on initial solution

SGD for Matrix Factorization

- Stochastic Gradient Descent as an alternative
- SGD on the objective $\mathcal{L} := \sum_{(u,i) \in S} (r_{ui} - \mathbf{q}_u \cdot \mathbf{p}_i^T)^2$
 - Pick a random user u and a random item i with rating r_{ui} (batch size 1)
 - Compute the gradients w.r.t. the parameters $\frac{\partial \mathcal{L}}{\partial \mathbf{q}_u}$ and $\frac{\partial \mathcal{L}}{\partial \mathbf{p}_i}$
 - Update the parameters $\mathbf{q}_u \leftarrow \mathbf{q}_u - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{q}_u}$, $\mathbf{p}_i \leftarrow \mathbf{p}_i - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{p}_i}$
 - η - learning rate
- In this case the gradient update step is rather simple
 - $e_{ui} \leftarrow r_{ui} - \mathbf{q}_u \cdot \mathbf{p}_i^T$ \\helper variable, the current error
 - $\mathbf{q}_u \leftarrow \mathbf{q}_u + 2\eta(e_{ui}\mathbf{p}_i)$
 - $\mathbf{p}_i \leftarrow \mathbf{p}_i + 2\eta(e_{ui}\mathbf{q}_u)$

Rating Prediction

- How to estimate the missing rating of user u for item i ?

Users	Items											
	1		3			5			5		4	
			5	4	?		4			2	1	3
	2	4		1			3		4	3	5	
		2	4		5			4			2	
			4	3	4	2					2	5
	1		3		3			2			4	

R

$$\hat{r}_{ui} = \mathbf{q}_u \cdot \mathbf{p}_i^T = \sum_k \mathbf{q}_{uk} \cdot \mathbf{p}_{ik}$$

$\mathbf{q}_u$ row u of Q
 $\mathbf{p}_i$ row i of P

Users	Factors			•	Items											
	.1	-.4	.2		1.1	-.2	.3	.5	-.2	-.5	.8	-.4	.3	1.4	2.4	-.9
	-.5	.6	.5		-.8	.7	.5	1.4	.3	-1	1.4	2.9	-.7	1.2	-.1	1.3
	-.2	.3	.5		2.1	-.4	.6	1.7	2.4	.9	-.3	.4	.8	.7	-.6	.1
	1.1	2.1	.3													
	-.7	2.1	-.2													
	-1	.7	.3													

Q P^T

Rating Prediction

- How to estimate the missing rating of user u for item i ?

Users	Items											
	1		3		?	5			5		4	
			5	4			4			2	1	3
	2	4		1			3		4	3	5	
		2	4		5			4			2	
			4	3	4	2					2	5
	1		3		3			2			4	

R

$$\hat{r}_{ui} = \mathbf{q}_u \cdot \mathbf{p}_i^T = \sum_k \mathbf{q}_{uk} \cdot \mathbf{p}_{ik}$$

$\mathbf{q}_u$ row u of Q
 $\mathbf{p}_i$ row i of P

Users	Factors		
	.1	-.4	.2
	-.5	.6	.5
	-.2	.3	.5
	1.1	2.1	.3
	-.7	2.1	-2
	-1	.7	.3

Q

•

Factors	Items											
	1.1	-.2	.3	.5	-.2	-.5	.8	-.4	.3	1.4	2.4	-.9
	-.8	.7	.5	1.4	.3	-1	1.4	2.9	-.7	1.2	-.1	1.3
	2.1	-.4	.6	1.7	2.4	.9	-.3	.4	.8	.7	-.6	.1

P^T

Rating Prediction

- How to estimate the missing rating of user u for item i ?

Users	Items											
	1		3			5			5		4	
			5	4	2.4		4			2	1	3
	2	4		1			3		4	3	5	
		2	4		5			4			2	
			4	3	4	2					2	5
	1		3		3			2			4	

R

$$\hat{r}_{ui} = \mathbf{q}_u \cdot \mathbf{p}_i^T = \sum_k \mathbf{q}_{uk} \cdot \mathbf{p}_{ik}$$

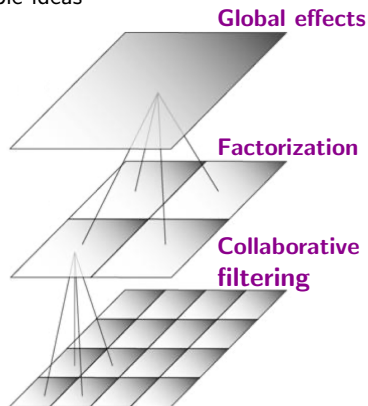
$\mathbf{q}_u$ row u of Q
 $\mathbf{p}_i$ row i of P

Users	Factors			Factors	Items											
	.1	-.4	.2		1.1	-.2	.3	.5	-.2	-.5	.8	-.4	.3	1.4	2.4	-.9
	-.5	.6	.5		-.8	.7	.5	1.4	.3	-1	1.4	2.9	-.7	1.2	-.1	1.3
	-.2	.3	.5		2.1	-.4	.6	1.7	2.4	.9	-.3	.4	.8	.7	-.6	.1
	1.1	2.1	.3													
	-.7	2.1	-.2													
	-1	.7	.3													

Q P^T

Side note: BellKor Recommender System

- The winner of the Netflix Challenge uses matrix factorization as **one** building block
 - The overall model is a combination of multiple ideas
- Multi scale modeling of the data:
Combine top level, 'regional' modeling of the data, with a refined, local view:
 - **Global:**
 - Overall deviations of users/movies
 - **Factorization:**
 - Addressing "regional" effects
 - **Collaborative filtering:**
 - Extract local patterns



User and Item Biases

- Certain users might give overly optimistic or pessimistic rating
- Certain movies tend to always have low ratings
- We introduce additional bias terms to capture these effects
 - Each user u can have a bias term b_u
 - Each item i can have a bias term b_i
 - Additional global bias term b shared by everyone
- The resulting optimization problem can be easily solved with SGD

$$\min_{\substack{\mathbf{P}, \mathbf{Q} \\ b_u, b_i, b}} \sum_{(u,i) \in S} (r_{ui} - (\mathbf{q}_u \cdot \mathbf{p}_i^T + b_u + b_i + b))^2$$

- The **cold start** problem is another important issue

Chapter: Dimensionality Reduction & Matrix Factorization

1. Introduction
2. Principal Component Analysis (PCA)
3. Singular Value Decomposition (SVD)
4. **Matrix Factorization**
 - Motivation & Approach
 - **Regularization & Sparsity**
 - Further Factorization Models
5. Neighbor Graph Methods
6. Autoencoders (Nonlinear Dimensionality Reduction)

Challenge: Overfitting

- Final Goal: Minimize SSE for **unseen test data**
- Proxy: Minimize SSE on **training data**
 - Want large k (number of factors) to capture all the signals
 - But, SSE on **test data** begins to rise for larger k
 - Why?
- Classical example of overfitting
 - With too many degrees freedom (too many free parameters) the model starts fitting noise
 - The model fits too well the training data but does not generalize well to unseen test data

Challenge: Overfitting

- Problem can easily be seen in our scenario:
 - In each step of the alternating optimization we solve an OLS regression
 - Number of regression parameters = k = number of latent factors
 - Number of data points used for regression = cardinality of $S_{*,i}/S_{u,*}$
 - Lots of parameters but not enough data points $\rightarrow$ regression can overfit
 - If k is large this might even lead to an **underdetermined** system of equations
 - Problem is ill posed
- Solution: **Regularization**
 - Interpretation for underdetermined systems: "In the absence of any other information, the parameter vector should be small (i.e. only small effect of features)"
 - Equivalently: We put a prior on the weights

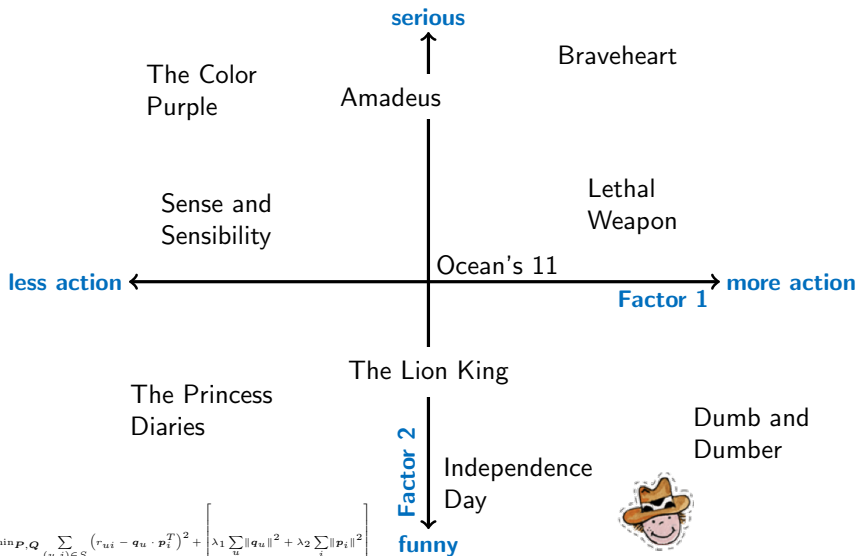
Regularization

- To solve overfitting we introduce regularization:
 - Allow rich model where we have sufficient data
 - Shrink aggressively where data are scarce

$$\min_{\mathbf{P}, \mathbf{Q}} \underbrace{\sum_{(u,i) \in S} \left(r_{ui} - \mathbf{q}_u \cdot \mathbf{p}_i^T \right)^2}_{\text{"error"}} + \underbrace{\left[\lambda_1 \sum_u \|\mathbf{q}_u\|^2 + \lambda_2 \sum_i \|\mathbf{p}_i\|^2 \right]}_{\text{"length"}}$$

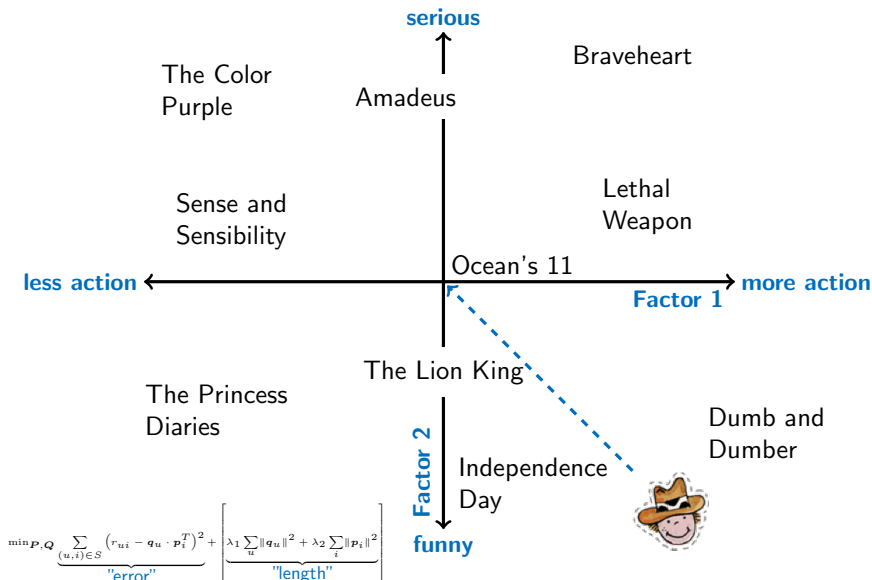
- λ_1 and λ_2 are user-defined regularization parameters
- Note: We do not care about the actual value of the objective function, but we care about the $\mathbf{P}, \mathbf{Q}$ that achieve the minimum of the objective

The Effect of Regularization

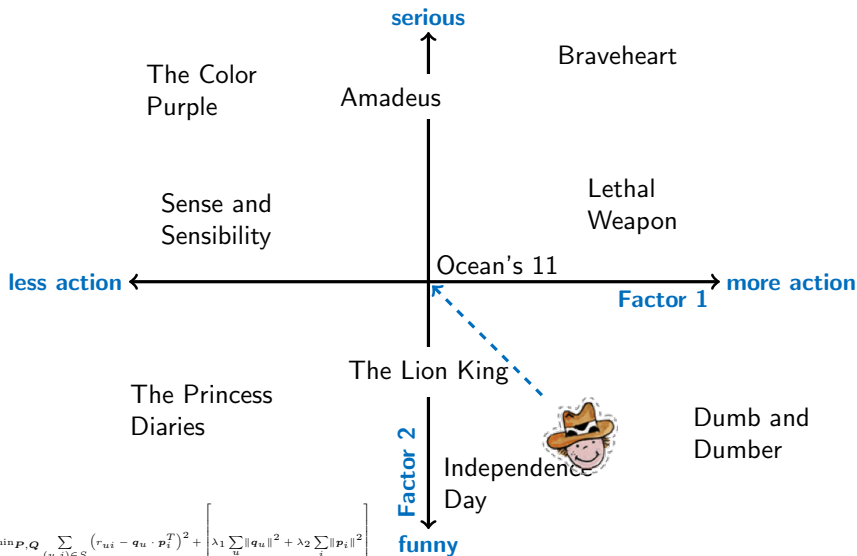


$$\min_{\mathbf{P}, \mathbf{Q}} \underbrace{\sum_{(u,i) \in S} (r_{ui} - \mathbf{q}_u \cdot \mathbf{p}_i^T)^2}_{\text{"error"}} + \underbrace{\left[\lambda_1 \sum_u \|\mathbf{q}_u\|^2 + \lambda_2 \sum_i \|\mathbf{p}_i\|^2 \right]}_{\text{"length"}}$$

The Effect of Regularization

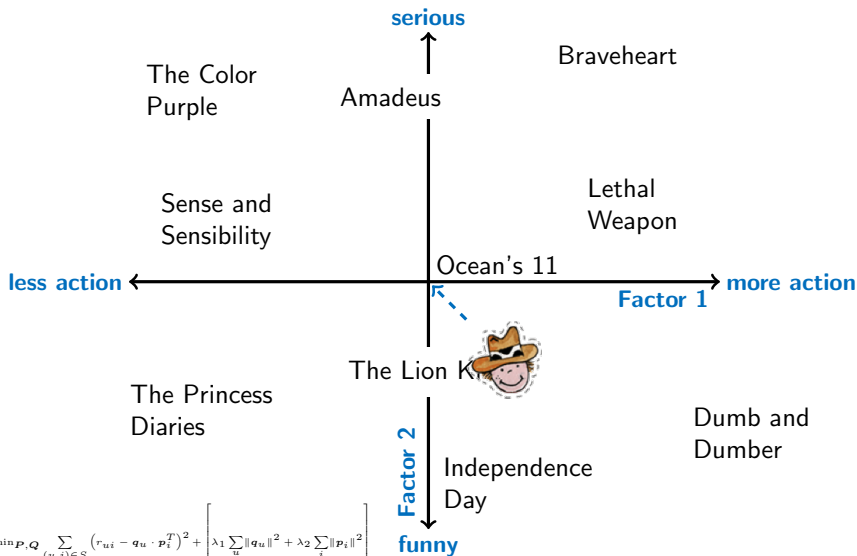


The Effect of Regularization



$$\min_{\mathbf{P}, \mathbf{Q}} \underbrace{\sum_{(u,i) \in S} (r_{ui} - \mathbf{q}_u \cdot \mathbf{p}_i^T)^2}_{\text{"error"}} + \underbrace{\left[\lambda_1 \sum_u \|\mathbf{q}_u\|^2 + \lambda_2 \sum_i \|\mathbf{p}_i\|^2 \right]}_{\text{"length"}}$$

The Effect of Regularization



$$\min_{\mathbf{P}, \mathbf{Q}} \underbrace{\sum_{(u,i) \in S} (r_{ui} - \mathbf{q}_u \cdot \mathbf{p}_i^T)^2}_{\text{"error"}} + \underbrace{\left[\lambda_1 \sum_u \|\mathbf{q}_u\|^2 + \lambda_2 \sum_i \|\mathbf{p}_i\|^2 \right]}_{\text{"length"}}$$

Regularization in Our Use Case

- Regularized Problem:

$$\min_{\mathbf{P}, \mathbf{Q}} \sum_{(u,i) \in S} (r_{ui} - \mathbf{q}_u \cdot \mathbf{p}_i^T)^2 + \left[\lambda_1 \sum_u \|\mathbf{q}_u\|^2 + \lambda_2 \sum_i \|\mathbf{p}_i\|^2 \right]$$

- Recap of unregularized problem: In each iteration of the alternating optimization we solved an OLS regression problem
- For the regularized version this becomes **ridge regression**
 - $\min_{\mathbf{p}_i} \sum_{i \in S_{*,i}} (r_{ui} - \mathbf{q}_u \cdot \mathbf{p}_i^T)^2 + \lambda_2 \|\mathbf{p}_i\|^2$
 - Effect: parameter values are forced to become smaller
 - Large values that only capture noise are avoided
- You know how to solve this!
 - Closed form solution (see linear regression slides)
 - Gradient descent

SGD for MF with Regularization and Biases

- SGD on the objective

$$\mathcal{L} := \min_{\mathbf{P}, \mathbf{Q}, b_u, b_i, b} \sum_{(u,i) \in S} \left(r_{ui} - (\mathbf{q}_u \cdot \mathbf{p}_i^T + b_u + b_i + b) \right)^2 + \left[\lambda_1 \sum_u \|\mathbf{q}_u\|^2 + \lambda_2 \sum_u \|\mathbf{q}_u\|^2 \right]$$

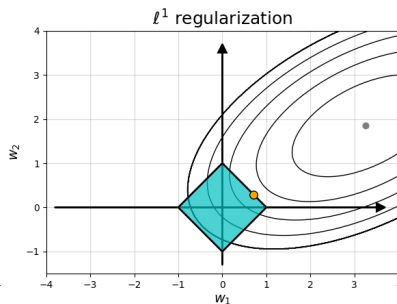
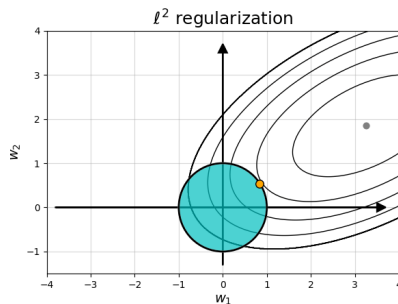
- Pick a random user u and a random item i with rating r_{ui} (batch size 1)
- The gradient update step is very similar to before
 - $e_{ui} \leftarrow r_{ui} - \mathbf{q}_u \cdot \mathbf{p}_i^T$ \\ helper variable, the current error
 - $\mathbf{q}_u \leftarrow \mathbf{q}_u + 2\eta (e_{ui}\mathbf{p}_i - \lambda_1 \mathbf{q}_u)$
 - $\mathbf{p}_i \leftarrow \mathbf{p}_i + 2\eta (e_{ui}\mathbf{q}_u - \lambda_2 \mathbf{p}_i)$
 - $b_u \leftarrow b_u + 2\eta e_{ui}$
 - $b_i \leftarrow b_i + 2\eta e_{ui}$
 - Usually directly set b to the global mean $b = \frac{1}{|S|} \sum_{(u,i) \in S} r_{ui}$

L2 vs. L1 Regularization

- Comparison: L2 vs. L1 regularization
 - L2 tries to "shrink" the parameter vector w **equally**
 - Large values are highly penalized due to the square in the L2 norm
 - Unlikely that any component will be **exactly 0**
 - L1 allows large values in individual components of w by shrinking other components to 0
 - L1 is suited to enforce **sparsity** of the parameter vector
- Why sparsity
 - Better interpretation
 - Only few values contribute to result
 - Unintuitive that **sparse input data** is generated based on **dense signal**
 - Less storage, faster processing

L2 vs. L1 Intuition

- An L1 constraint promotes sparsity



by @itayevron

Chapter: Dimensionality Reduction & Matrix Factorization

1. Introduction
2. Principal Component Analysis (PCA)
3. Singular Value Decomposition (SVD)
4. **Matrix Factorization**
 - Motivation & Approach
 - Regularization & Sparsity
 - **Further Factorization Models**
5. Neighbor Graph Methods
6. Autoencoders (Nonlinear Dimensionality Reduction)

Further Factorization Models

- Matrix factorization is extremely powerful
 - Dimensionality reduction, data analysis/data understanding, prediction of missing values, . . .
- Various extensions have been proposed
 - Enforcing different constraints or operating on different data types
 - Important goal: better interpretation of result
- Non Negative Matrix Factorization (next slides)
- Boolean Matrix Factorization
 - Factorize Boolean A in Boolean Q and P

Non-Negative Matrix Factorization

- Often data is given in form of non-negative values
 - Rating values between 1 to 5; income, age, ... of persons; number of words in a document; grayscale images; etc.
- However: SVD (and the other approaches presented before) might lead to factors containing **negative values**
 - Difficult to interpret: non-negative data is "generated" based on negative factors; what do these negative factors mean?
 - Predicted values might also become negative
- Solution: Non-Negative Matrix Factorization

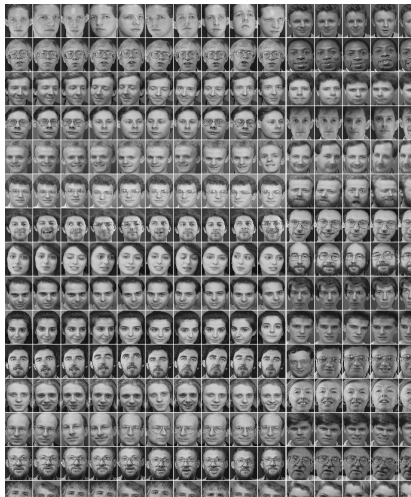
Non-Negative Matrix Factorization

- Task: Factorize non negative A in non negative Q and P , i.e.
 $A \approx Q \cdot P^T$
- Formally:
 - Given $A \in \mathbb{R}^{n \times d}$ with $A_{ij} \geq 0$ and integer k , find $P \in \mathbb{R}^{n \times k}$, $Q \in \mathbb{R}^{k \times d}$ such that $\|A - Q \cdot P^T\|_F$ is minimized subject to $Q \geq 0$ and $P \geq 0$

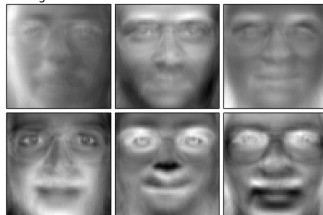
$$\min_{P \geq 0, Q \geq 0} \|A - Q \cdot P^T\|_F$$

- Constrained optimization

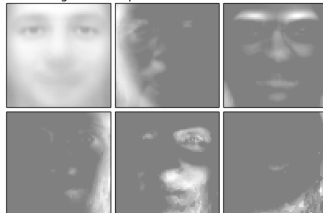
NNMF Example: Olivetti Faces Data



Eigenfaces - RandomizedPCA - Train time 1.4s



Non-negative components - NMF - Train time 2.9s

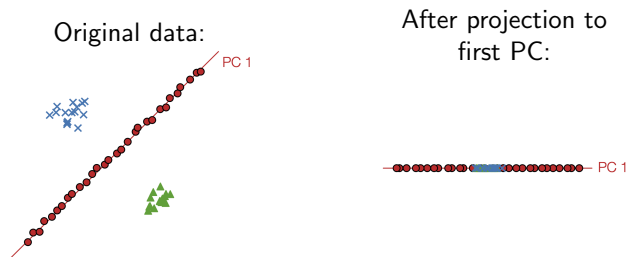


Chapter: Dimensionality Reduction & Matrix Factorization

1. Introduction
2. Principal Component Analysis (PCA)
3. Singular Value Decomposition (SVD)
4. Matrix Factorization
5. **Neighbor Graph Methods**
6. Autoencoders (Nonlinear Dimensionality Reduction)

Preserving global vs. preserving local similarity

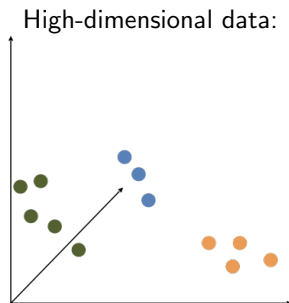
- PCA tries to find a global structure
 - Can lead to local inconsistencies
 - Far away point can become nearest neighbors
- Illustration:



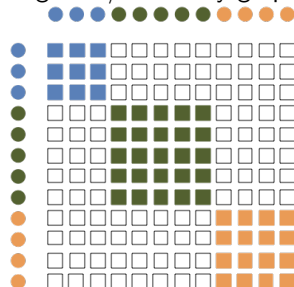
- Idea: Preserve local structure instead
- Example: <https://projector.tensorflow.org/>

Neighbor Graph Methods

- All methods we have seen so far are based on **matrix factorizations**
- An alternative class of methods based on **neighbor graphs**



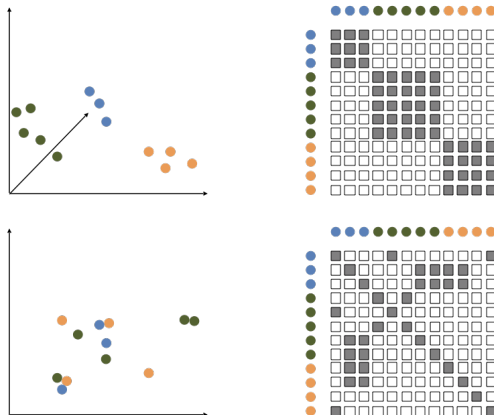
Neighbor / similarity graph:



- Idea: Low-dimensional neighborhood similar to the original neighborhood

Neighbor Graph Methods

1. Construct neighbor graph of high dimensional data
2. Initialize points in low dimensional space
3. Optimize coordinates in low dimensional space s.t. similarities align



tSNE: t Distributed Stochastic Neighbor Embedding

- High dimensional similarities for input $\mathbf{x}_i$:

$$p_{j|i} = \frac{\exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma_i^2}\right)}{\sum_{k \neq i} \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_k\|^2}{2\sigma_i^2}\right)} \quad p_{ii} = 0, \quad p_{ij} = p_{j|i} + p_{i|j}$$

- σ_i chosen to achieve fixed perplexity (effective number of neighbors)
- Low dimensional similarities for (to be optimized) parameters $\mathbf{y}_i$:

$$q_{i,j} = \frac{(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}}{\sum_k \sum_{l \neq k} (1 + \|\mathbf{y}_k - \mathbf{y}_l\|^2)^{-1}} \quad q_{ii} = 0$$

- Minimize the KL divergence:

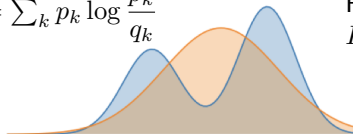
$$\min_{\mathbf{y}_i} KL(P||Q) = \sum_i \sum_{j \neq i} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

Note on the KL Divergence

- $KL(P||Q) \geq 0$ for any P and Q , $KL(P||Q) = 0$ iff $P = Q$
- The KL Divergence is asymmetric $KL(P||Q) \neq KL(Q||P)$
 - Example: Given P we are optimizing over Q

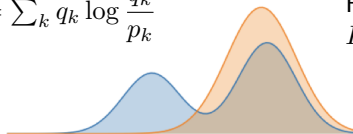
- $KL(P||Q) = \sum_k p_k \log \frac{p_k}{q_k}$

Forward KL is Mean Seeking
 $KL(P||Q)$

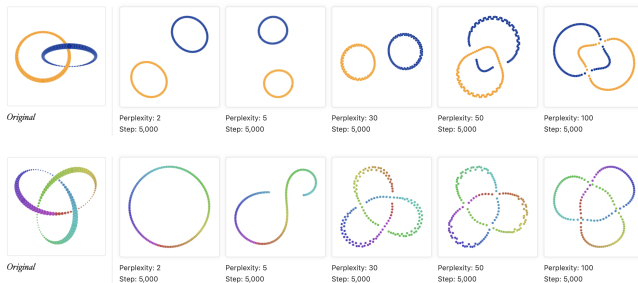


- $KL(Q||P) = \sum_k q_k \log \frac{q_k}{p_k}$

Reverse KL is Mode Seeking
 $KL(Q||P)$



Interactive tSNE



Source of illustration, for more details and interactive widgets

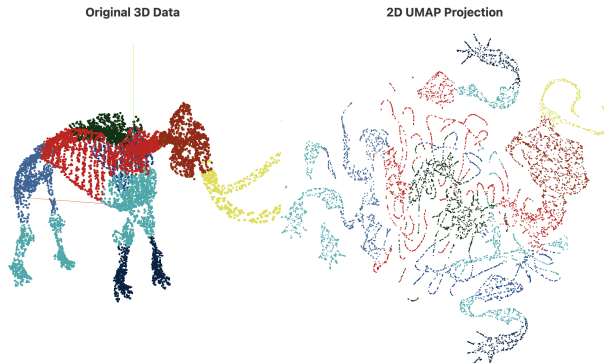
The Advantages and Disadvantages of t-SNE

- The current standard for visualizing high-dimensional data
- Helps understand "black box" algorithms like DNN
- Reduced "crowding problem" with heavy tailed distribution

- t-SNE plots can sometimes be mysterious or misleading
 - Be careful with cluster sizes and cluster distances
- Can be sensitive to hyperparameters
 - Random noise does not always look random
- No easy way to compute the embedding of new data
- Not great for more than 3 dimensions

Uniform Manifold Approximation and Projection

- Principled approach relying on topological spaces, category theory, ...



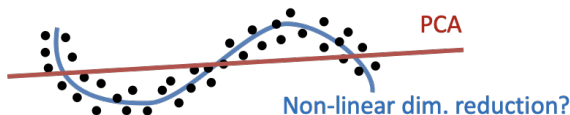
[Link for details, widgets, a comparison to T SNE, ...](#)

Chapter: Dimensionality Reduction & Matrix Factorization

1. Introduction
2. Principal Component Analysis (PCA)
3. Singular Value Decomposition (SVD)
4. Matrix Factorization
5. Neighbor Graph Methods
6. **Autoencoders (Nonlinear Dimensionality Reduction)**

Motivation

- PCA / SVD can only capture linear structure (linear variations in the data)
 - Recall: transformed data is $\mathbf{Y} = \tilde{\mathbf{X}} \cdot \mathbf{\Gamma}$
 - Linear projection by the eigenvectors $\mathbf{\Gamma}$
- However, data often lies on a non-linear low dimensional manifold



- Idea: find a non-linear projection of the data

Autoencoders

- An **autoencoder** is a neural network that:
 - finds a compact representation of the data
 - by learning to reconstruct its input

$$f(\mathbf{x}, \mathbf{W}) := \hat{\mathbf{x}} \approx \mathbf{x}$$

- Goal: minimize the reconstruction error between $\hat{\mathbf{x}}$ and $\mathbf{x}$

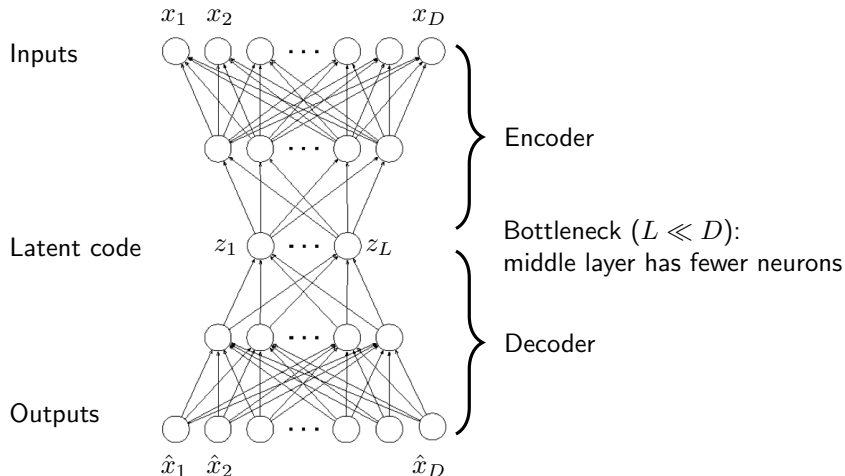
$$\min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N |f(x_i, \mathbf{W}) - x_i|^2$$

- Alternative view: find a **latent representation** $\mathbf{z} \in \mathbb{R}^L$ which is a compact code for $\mathbf{x} \in \mathbb{R}^D$ since $L \ll D$
 - $f_{enc}(\mathbf{x}) = \mathbf{z}$ # encoder: project the data to a lower dimension
 - $f_{dec}(\mathbf{z}) \approx \mathbf{x}$ # decoder: reconstruct the data from the latent code

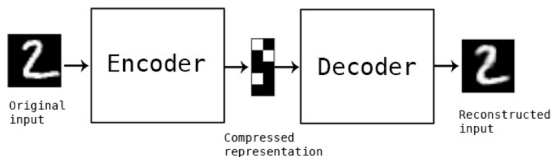
$$f_{dec}(f_{enc}(\mathbf{x})) \approx \mathbf{x}$$

Autoencoders

- f_{enc} and f_{dec} are non-linear functions implemented by a neural network



Autoencoders: Example



Original image:



Reconstruction:



Autoencoders

- An autoencoder whose code dimension is less than the input dimension ($L \ll D$) is called **undercomplete**
 - Forces the autoencoder to capture the most salient features of the data
 - $L \geq D$ (**overcomplete**) the autoencoder can simply learn the identity function
- Training autoencoders in practice:
 - Add a regularization term to the SSE loss to prevent overfitting
 - Weight tying: f_{enc} and f_{dec} share the same weights
- Other extensions:
 - Denoising autoencoders (DAEs): receive a corrupted (noisy) training data point as input and predict the "clean" uncorrupted data point as output
 - Variational autoencoders (covered in our DGM lecture)

(Linear) Autoencoders & PCA: Comparison

- What if f_{enc} and f_{dec} are linear functions?

- $f_{enc}(\mathbf{x}, \mathbf{W}_1) = \mathbf{x}\mathbf{W}_1$

- $f_{dec}(\mathbf{z}, \mathbf{W}_2) = \mathbf{z}\mathbf{W}_2$

$$\#\mathbf{W}_1 \in \mathbb{R}^{D \times L}$$

$$\#\mathbf{W}_2 \in \mathbb{R}^{L \times D}$$

- We have: $f_{dec}(f_{enc}(\mathbf{x})) = \mathbf{x}\mathbf{W}_1\mathbf{W}_2$ and

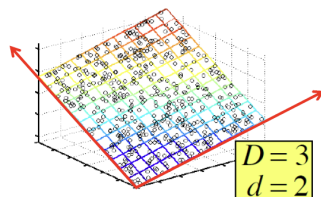
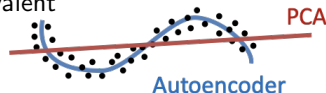
$$\min_{\mathbf{W}_1 \mathbf{W}_2} \frac{1}{N} \sum_{i=1}^N |f(\mathbf{x}_i, \mathbf{W}) - \mathbf{x}_i|^2 = \min_{\mathbf{W}_1 \mathbf{W}_2} \frac{1}{N} \sum_{i=1}^N |\mathbf{x}_i \mathbf{W}_1 \mathbf{W}_2 - \mathbf{x}_i|^2 =$$

$$\underbrace{\min_{\mathbf{W}} \frac{1}{N} \sum_{i=1}^N |\mathbf{x}_i \mathbf{W} - \mathbf{x}_i|^2}_{\text{Equivalent: } \mathbf{W}_1 \mathbf{W}_2 = \mathbf{W} \text{ s.t rank of } \mathbf{W} \text{ is } L}$$

- Optimal solution (assuming normalization):
 - PCA: $\mathbf{W}^* = \mathbf{\Gamma}$ where $\mathbf{\Gamma}$ are the (top L) eigenvectors of $\mathbf{X}^T \mathbf{X}$

Summary

- Dimensionality Reduction and Matrix Factorization are highly related
 - And can be used for various purposes
- PCA, SVD (and linear Autoencoders) are equivalent
 - Optimal low-rank approximation
- Matrix factorization for rating prediction
 - Very general formulation: allows to handle, e.g., missing entries
- Autoencoders and t-SNE perform non-linear dimensionality reduction
- Why are these techniques useful?
 - Less storage required
 - More efficient processing of the data
 - Remove redundant and noisy features
 - Discover hidden correlations/topics/concepts
 - Interpretation and visualization



Reading material

- Bishop, chapters: 12.1 , 12.2.1
- Leskovec, Rajaraman, Ullman- Mining of Massive Datasets: chapter 11
- Goodfellow- Deep Learning: chapter 14
- t-SNE
- Understanding UMAP